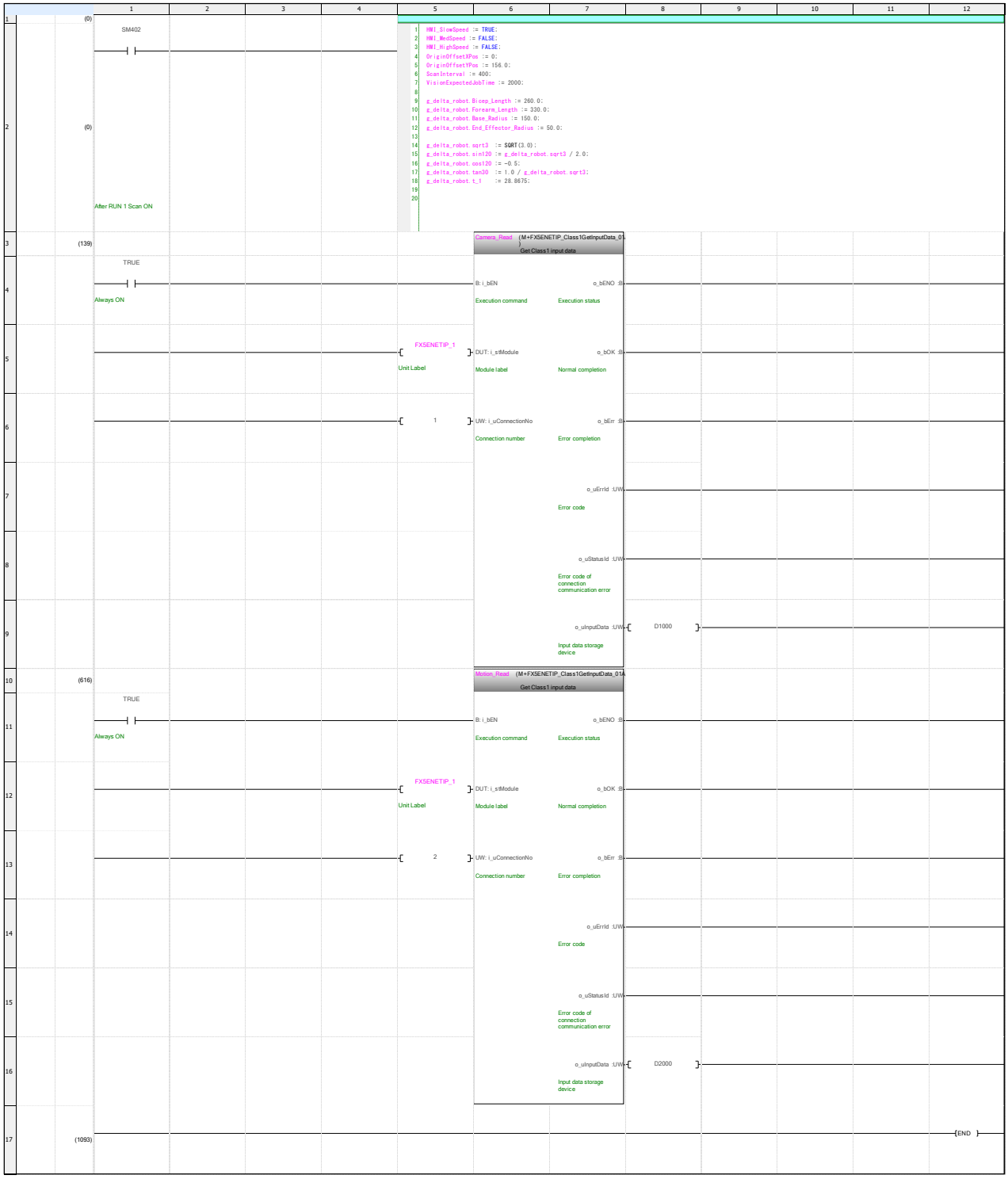

SM402
After RUN 1 Scan ON
HMI_SlowSpeed := TRUE;
HMI_MedSpeed := FALSE;
HMI_HighSpeed := FALSE;
OriginOffsetXPos := 0;
OriginOffsetYPos := 156.0;
ScanInterval := 400;
VisionExpectedJobTime := 2000;
g_delta_robot_Bicep_Length := 260.0;
g_delta_robot_Forearm_Length := 330.0;
g_delta_robot_Base_Radius := 150.0;
g_delta_robot_End_Effector_Radius := 50.0;
g_delta_robot_sqrt3 := SQRT(3.0);
g_delta_robot_sin120 := g_delta_robot_sqrt3 / 2.0;
g_delta_robot_cos120 := -0.5;
g_delta_robot_tan30 := 1.0 / g_delta_robot_sqrt3;
g_delta_robot_t_1 := 28.8675;
Camera_Read
(M+FXSENETIP_Class1GetInputData_01)
Get Class1 input data
TRUE
Always ON
B: i_bEN
Execution command
o_bENO :B
Execution status
FXSENETIP_1
Unit Label
DUT: i_stModule
Module label
o_bOK :B
Normal completion
1
UW: i_uConnectionNo
Connection number
o_bErr :B
Error completion
o_uErrId :UW
Error code
o_uStatusId :UW
Error code of connection communication error
o_uInputData :UW
D1000
Input data storage device
Motion_Read
(M+FXSENETIP_Class1GetInputData_01A)
Get Class1 input data
TRUE
Always ON
B: i_bEN
Execution command
o_bENO :B
Execution status
FXSENETIP_1
Unit Label
DUT: i_stModule
Module label
o_bOK :B
Normal completion
2
UW: i_uConnectionNo
Connection number
o_bErr :B
Error completion
o_uErrId :UW
Error code
o_uStatusId :UW
Error code of connection communication error
o_uInputData :UW
D2000
Input data storage device
END

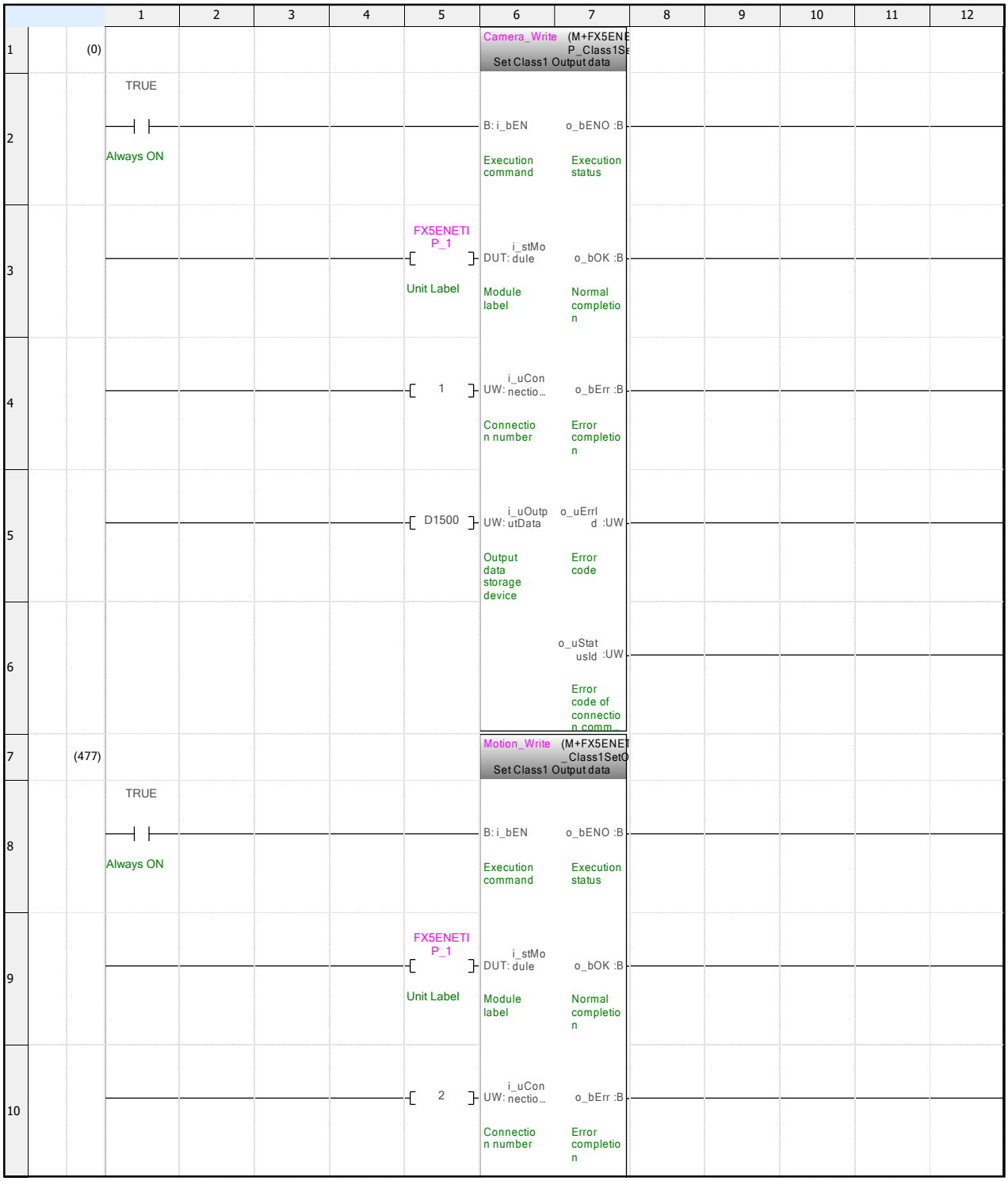
Inline Structured Text: EIP_Read (Row2)

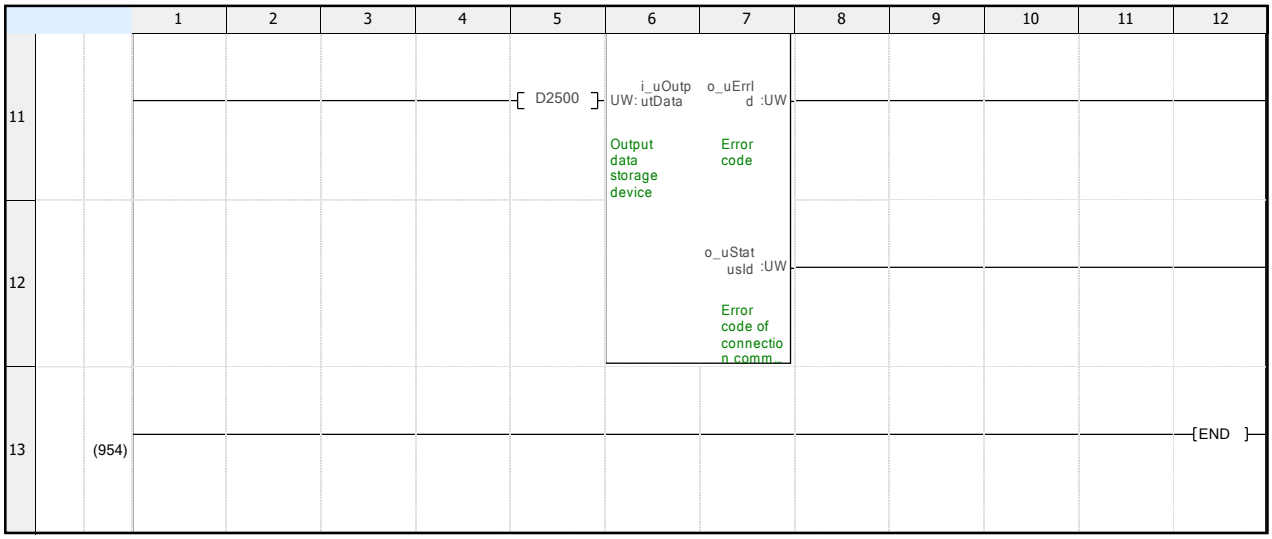
```
HMI_SlowSpeed := TRUE;
HMI_MedSpeed := FALSE;
HMI_HighSpeed := FALSE;
OriginOffsetXPos := 0;
OriginOffsetYPos := 156.0;
ScanInterval := 400;
VisionExpectedJobTime := 2000;

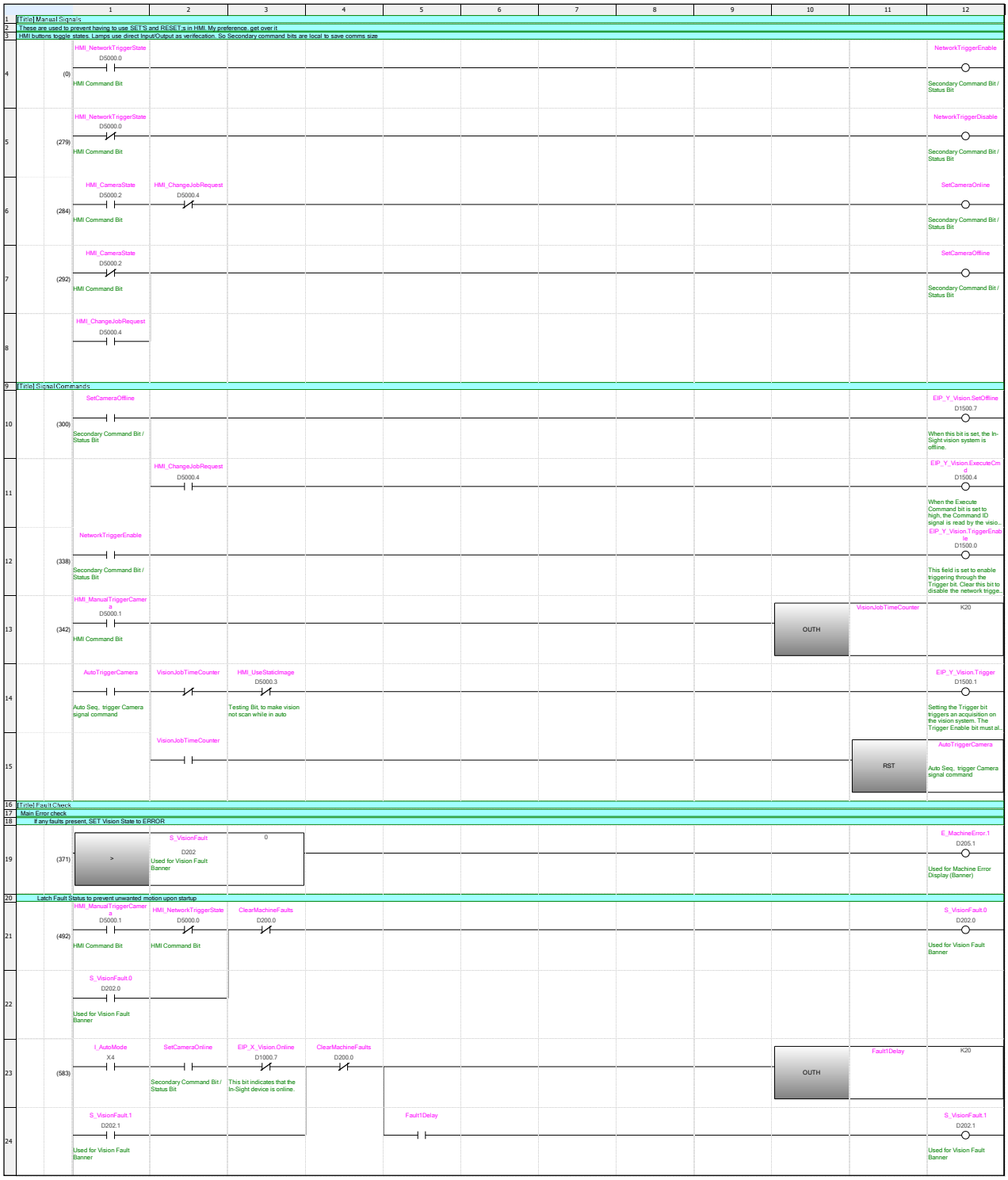
g_delta_robot.Bicep_Length := 260.0;
g_delta_robot.Forearm_Length := 330.0;
g_delta_robot.Base_Radius := 150.0;
g_delta_robot.End_Effector_Radius := 50.0;

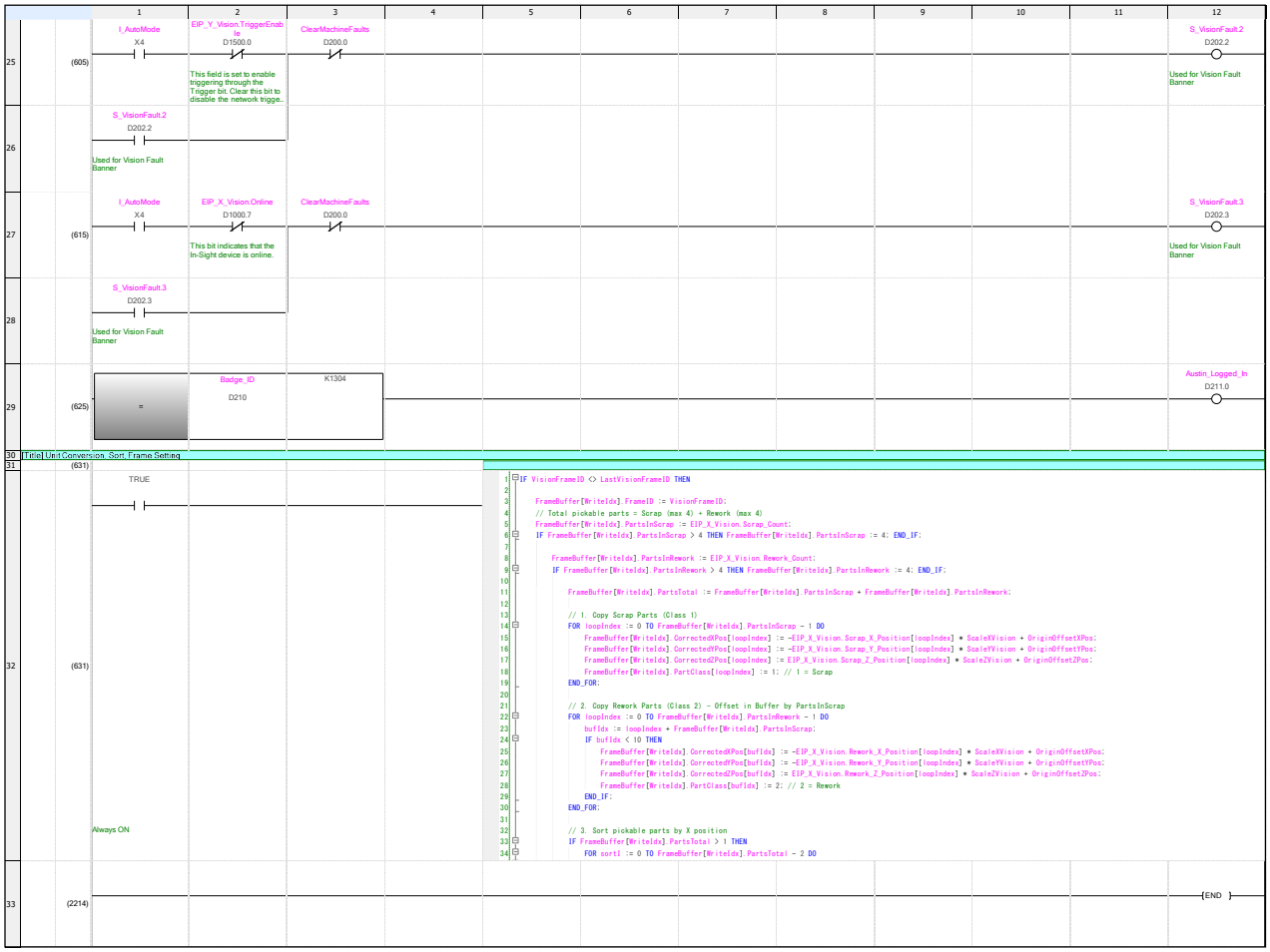
g_delta_robot.sqrt3  := SQRT(3.0);
g_delta_robot.sin120 := g_delta_robot.sqrt3 / 2.0;
g_delta_robot.cos120 := -0.5;
g_delta_robot.tan30  := 1.0 / g_delta_robot.sqrt3;
g_delta_robot.t_1    := 28.8675;


```


1
2
3
4
5
6
7
8
9
10
11
12
(0)
Camera_Write
(M+FX5ENE
P_Class1Se
Set Class1 Output data
TRUE
B: i_bEN
o_bENO :B
Always ON
Execution command
Execution status
FX5ENETI
P_1
i_stMo
DUT: dule
o_bOK :B
Unit Label
Module label
Normal completio n
i_uCon
UW: nectio…
o_bErr :B
1
Connectio n number
Error completio n
i_uOutp
UW: utData
o_uErrI d :UW
D1500
Output data storage device
Error code
o_uStat usId :UW
Error code of connectio n comm…
(477)
Motion_Write
(M+FX5ENET
_Class1SetO
Set Class1 Output data
TRUE
B: i_bEN
o_bENO :B
Always ON
Execution command
Execution status
FX5ENETI
P_1
i_stMo
DUT: dule
o_bOK :B
Unit Label
Module label
Normal completio n
i_uCon
UW: nectio…
o_bErr :B
2
Connectio n number
Error completio n


1
2
3
4
5
6
7
8
9
10
11
12
11
D2500
i_uOutp
UW: utData
o_uErrI
d :UW
Output
data
storage
device
Error
code
12
o_uStat
usId :UW
Error
code of
connectio
n comm...
13
(954)
END


[Title] Manual Signals
These are used to prevent having to use SETS and RESET;s in HMI. My preference, get over it
HMI buttons toggle states. Lamps use direct Input/Output as verification. So Secondary command bits are local to save comms size
HMI_NetworkTriggerState D5000.0
(0)
HMI Command Bit
NetworkTriggerEnable
Secondary Command Bit / Status Bit
HMI_NetworkTriggerState D5000.0
(279)
HMI Command Bit
NetworkTriggerDisable
Secondary Command Bit / Status Bit
HMI_CameraState D5000.2
HMI_ChangeJobRequest D5000.4
(284)
HMI Command Bit
SetCameraOnline
Secondary Command Bit / Status Bit
HMI_CameraState D5000.2
(292)
HMI Command Bit
SetCameraOffline
Secondary Command Bit / Status Bit
HMI_ChangeJobRequest D5000.4
[Title] Signal Commands
SetCameraOffline
(300)
Secondary Command Bit / Status Bit
EIP_Y_Vision.SetOffline D1500.7
When this bit is set, the In-Sight vision system is offline.
HMI_ChangeJobRequest D5000.4
EIP_Y_Vision.ExecuteCmd D1500.4
When the Execute Command bit is set to high, the Command ID signal is read by the visio...
NetworkTriggerEnable
(338)
Secondary Command Bit / Status Bit
EIP_Y_Vision.TriggerEnable D1500.0
This field is set to enable triggering through the Trigger bit. Clear this bit to disable the network trigge...
HMI_ManualTriggerCamera D5000.1
(342)
HMI Command Bit
OUTH
VisionJobTimeCounter
K20
AutoTriggerCamera
VisionJobTimeCounter
HMI_UseStaticImage D5000.3
Auto Seq, trigger Camera signal command
Testing Bit, to make vision not scan while in auto
EIP_Y_Vision.Trigger D1500.1
Setting the Trigger bit triggers an acquisition on the vision system. The Trigger Enable bit must al...
VisionJobTimeCounter
RST
AutoTriggerCamera
Auto Seq, trigger Camera signal command
[Title] Fault Check
Main Error check
If any faults present, SET Vision State to ERROR
(371)
>
S_VisionFault D202
Used for Vision Fault Banner
0
E_MachineError.1 D205.1
Used for Machine Error Display (Banner)
Latch Fault Status to prevent unwanted motion upon startup
HMI_ManualTriggerCamera D5000.1
HMI_NetworkTriggerState D5000.0
ClearMachineFaults D200.0
(492)
HMI Command Bit
HMI Command Bit
S_VisionFault.0 D202.0
Used for Vision Fault Banner
S_VisionFault.0 D202.0
Used for Vision Fault Banner
I_AutoMode X4
SetCameraOnline
EIP_X_Vision.Online D1000.7
ClearMachineFaults D200.0
(583)
Secondary Command Bit / Status Bit
This bit indicates that the In-Sight device is online.
OUTH
Fault1Delay
K20
S_VisionFault.1 D202.1
Used for Vision Fault Banner
Fault1Delay
S_VisionFault.1 D202.1
Used for Vision Fault Banner




```
IF VisionFrameID <> LastVisionFrameID THEN

    FrameBuffer[WriteIdx].FrameID := VisionFrameID;
    // Total pickable parts = Scrap (max 4) + Rework (max 4)
    FrameBuffer[WriteIdx].PartsInScrap := EIP_X_Vision.Scrap_Count;
    IF FrameBuffer[WriteIdx].PartsInScrap > 4 THEN FrameBuffer[WriteIdx].PartsInScrap := 4; END_IF;

        FrameBuffer[WriteIdx].PartsInRework := EIP_X_Vision.Rework_Count;
        IF FrameBuffer[WriteIdx].PartsInRework > 4 THEN FrameBuffer[WriteIdx].PartsInRework := 4; END_IF;

            FrameBuffer[WriteIdx].PartsTotal := FrameBuffer[WriteIdx].PartsInScrap + FrameBuffer[WriteIdx].PartsInRework;

            // 1. Copy Scrap Parts (Class 1)
            FOR loopIndex := 0 TO FrameBuffer[WriteIdx].PartsInScrap - 1 DO
                FrameBuffer[WriteIdx].CorrectedXPos[loopIndex] := -EIP_X_Vision.Scrap_X_Position[loopIndex] * ScaleXVision + OriginOffsetXPos;
                FrameBuffer[WriteIdx].CorrectedYPos[loopIndex] := -EIP_X_Vision.Scrap_Y_Position[loopIndex] * ScaleYVision + OriginOffsetYPos;
                FrameBuffer[WriteIdx].Corrected2Pos[loopIndex] := EIP_X_Vision.Scrap_Z_Position[loopIndex] * ScaleZVision + OriginOffsetZPos;
                FrameBuffer[WriteIdx].PartClass[loopIndex] := 1; // 1 = Scrap
            END_FOR;

            // 2. Copy Rework Parts (Class 2) - Offset in Buffer by PartsInScrap
            FOR loopIndex := 0 TO FrameBuffer[WriteIdx].PartsInRework - 1 DO
                bufIdx := loopIndex + FrameBuffer[WriteIdx].PartsInScrap;
                IF bufIdx < 10 THEN
                    FrameBuffer[WriteIdx].CorrectedXPos[bufIdx] := -EIP_X_Vision.Rework_X_Position[loopIndex] * ScaleXVision + OriginOffsetXPos;
                    FrameBuffer[WriteIdx].CorrectedYPos[bufIdx] := -EIP_X_Vision.Rework_Y_Position[loopIndex] * ScaleYVision + OriginOffsetYPos;
                    FrameBuffer[WriteIdx].Corrected2Pos[bufIdx] := EIP_X_Vision.Rework_Z_Position[loopIndex] * ScaleZVision + OriginOffsetZPos;
                    FrameBuffer[WriteIdx].PartClass[bufIdx] := 2; // 2 = Rework
                END_IF;
            END_FOR;

            // 3. Sort pickable parts by X position
            IF FrameBuffer[WriteIdx].PartsTotal > 1 THEN
                FOR sortI := 0 TO FrameBuffer[WriteIdx].PartsTotal - 2 DO
```

Inline Structured Text: Vision (Row32)

```
IF VisionFrameID <> LastVisionFrameID THEN

    FrameBuffer[WriteIdx].FrameID := VisionFrameID;
    // Total pickable parts = Scrap (max 4) + Rework (max 4)
    FrameBuffer[WriteIdx].PartsInScrap := EIP_X_Vision.Scrap_Count;
    IF FrameBuffer[WriteIdx].PartsInScrap > 4 THEN FrameBuffer[WriteIdx].PartsInScrap := 4; END_IF;

        FrameBuffer[WriteIdx].PartsInRework := EIP_X_Vision.Rework_Count;
        IF FrameBuffer[WriteIdx].PartsInRework > 4 THEN FrameBuffer[WriteIdx].PartsInRework := 4;
END_IF;

            FrameBuffer[WriteIdx].PartsTotal := FrameBuffer[WriteIdx].PartsInScrap +
FrameBuffer[WriteIdx].PartsInRework;

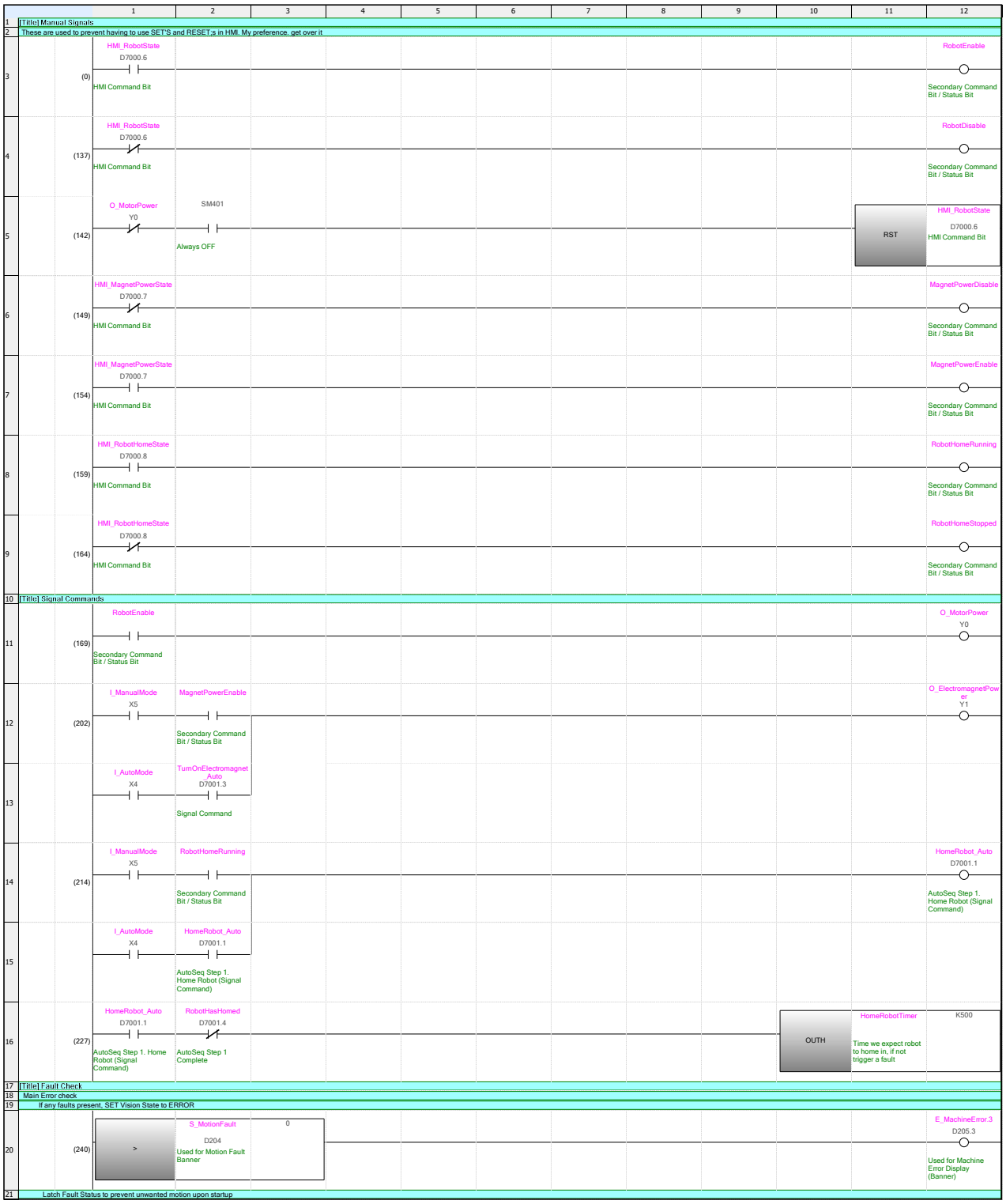
            // 1. Copy Scrap Parts (Class 1)
            FOR loopIndex := 0 TO FrameBuffer[WriteIdx].PartsInScrap - 1 DO
                FrameBuffer[WriteIdx].CorrectedXPos[loopIndex] :=
-EIP_X_Vision.Scrap_X_Position[loopIndex] * ScaleXVision + OriginOffsetXPos;
                FrameBuffer[WriteIdx].CorrectedYPos[loopIndex] :=
-EIP_X_Vision.Scrap_Y_Position[loopIndex] * ScaleYVision + OriginOffsetYPos;
                FrameBuffer[WriteIdx].CorrectedZPos[loopIndex] :=
EIP_X_Vision.Scrap_Z_Position[loopIndex] * ScaleZVision + OriginOffsetZPos;
                FrameBuffer[WriteIdx].PartClass[loopIndex] := 1; // 1 = Scrap
            END_FOR;

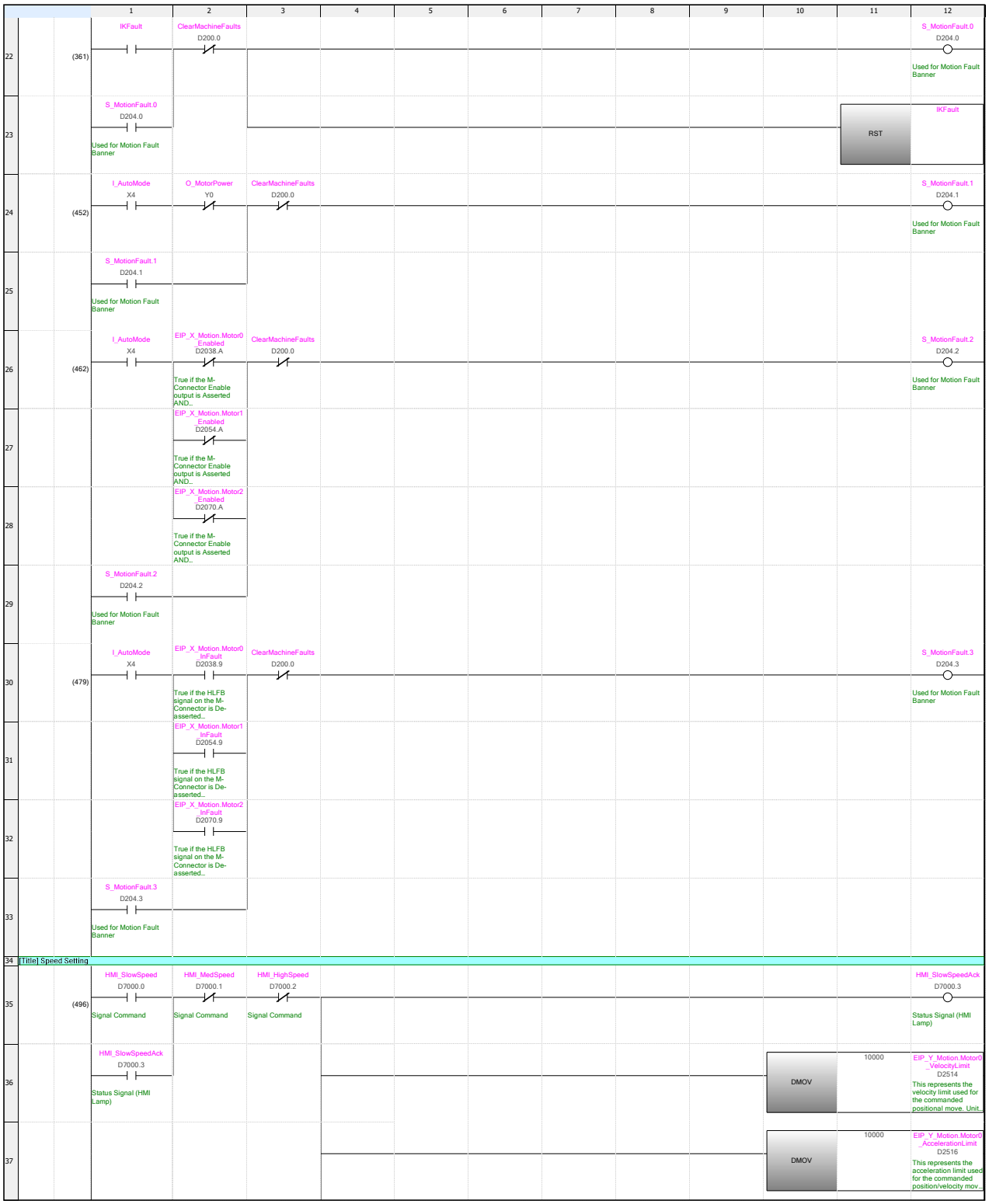
            // 2. Copy Rework Parts (Class 2) - Offset in Buffer by PartsInScrap
            FOR loopIndex := 0 TO FrameBuffer[WriteIdx].PartsInRework - 1 DO
                bufIdx := loopIndex + FrameBuffer[WriteIdx].PartsInScrap;
                IF bufIdx < 10 THEN
                    FrameBuffer[WriteIdx].CorrectedXPos[bufIdx] :=
-EIP_X_Vision.Rework_X_Position[loopIndex] * ScaleXVision + OriginOffsetXPos;
                    FrameBuffer[WriteIdx].CorrectedYPos[bufIdx] :=
-EIP_X_Vision.Rework_Y_Position[loopIndex] * ScaleYVision + OriginOffsetYPos;
                    FrameBuffer[WriteIdx].CorrectedZPos[bufIdx] :=
EIP_X_Vision.Rework_Z_Position[loopIndex] * ScaleZVision + OriginOffsetZPos;
                    FrameBuffer[WriteIdx].PartClass[bufIdx] := 2; // 2 = Rework
                END_IF;
            END_FOR;

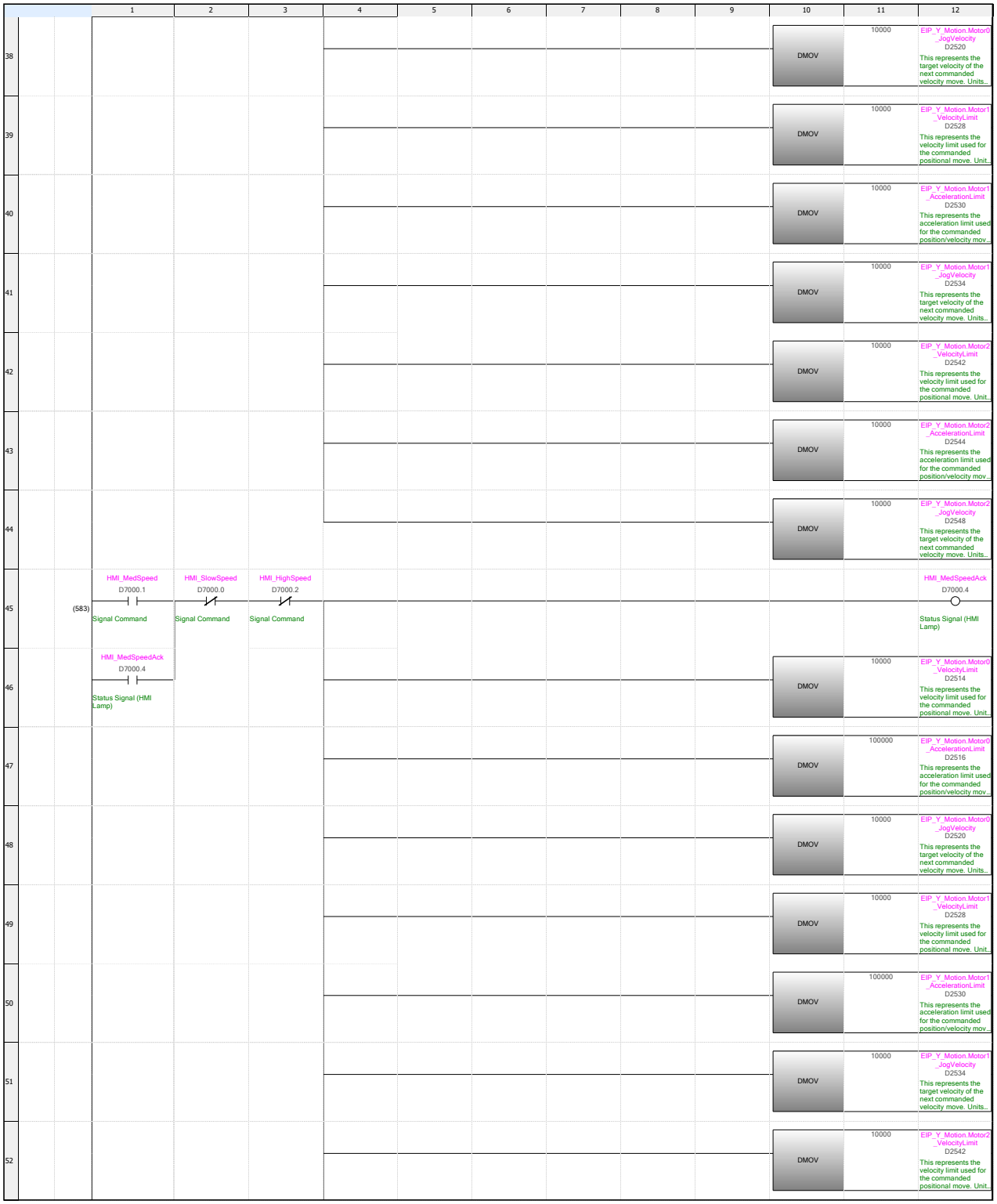
            // 3. Sort pickable parts by X position
            IF FrameBuffer[WriteIdx].PartsTotal > 1 THEN
                FOR sortI := 0 TO FrameBuffer[WriteIdx].PartsTotal - 2 DO
                    FOR sortJ := 0 TO FrameBuffer[WriteIdx].PartsTotal - sortI - 2 DO
                        IF FrameBuffer[WriteIdx].CorrectedXPos[sortJ] <
FrameBuffer[WriteIdx].CorrectedXPos[sortJ + 1] THEN
                            // Swap X
```

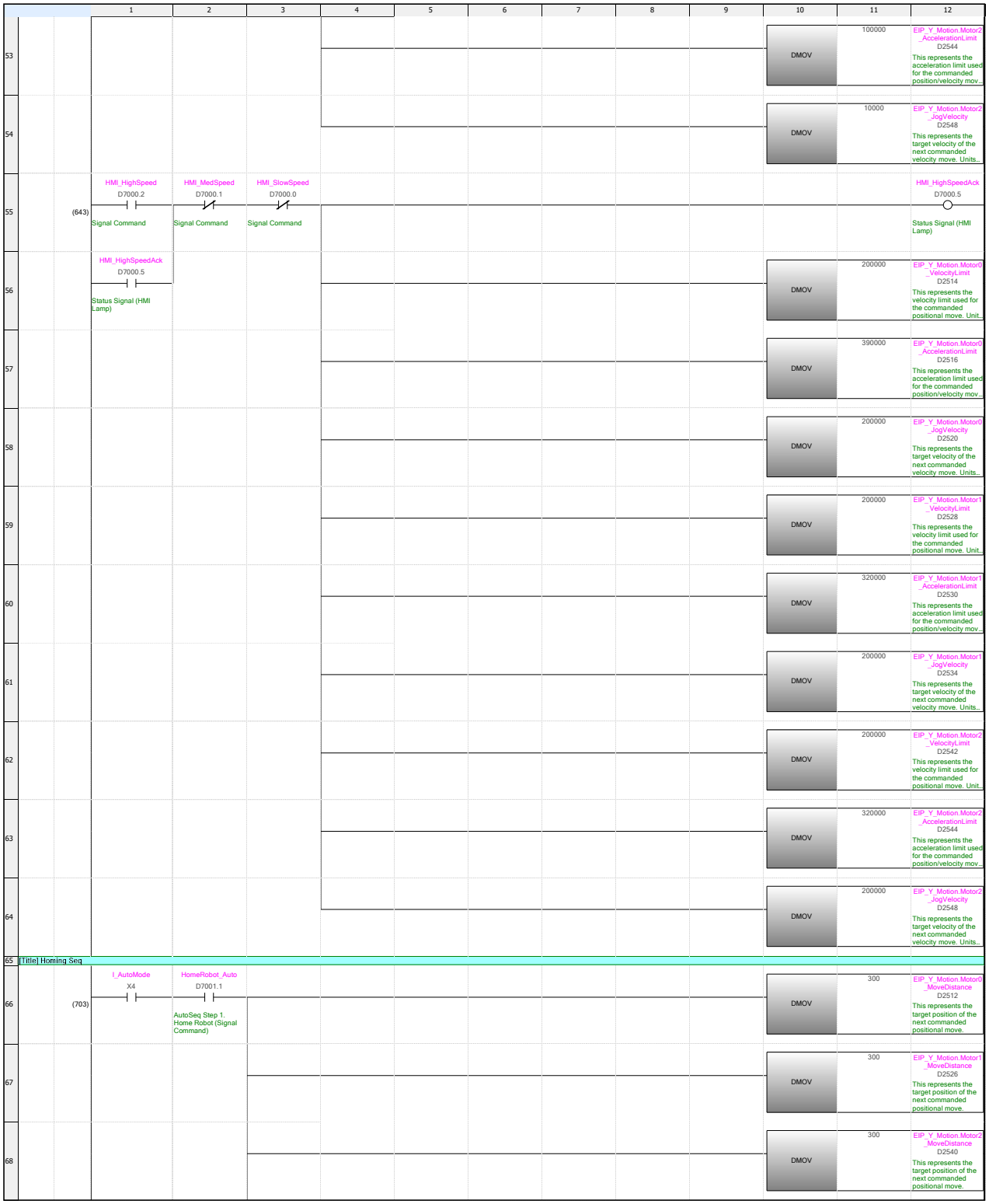
Inline Structured Text: Vision (Row32)

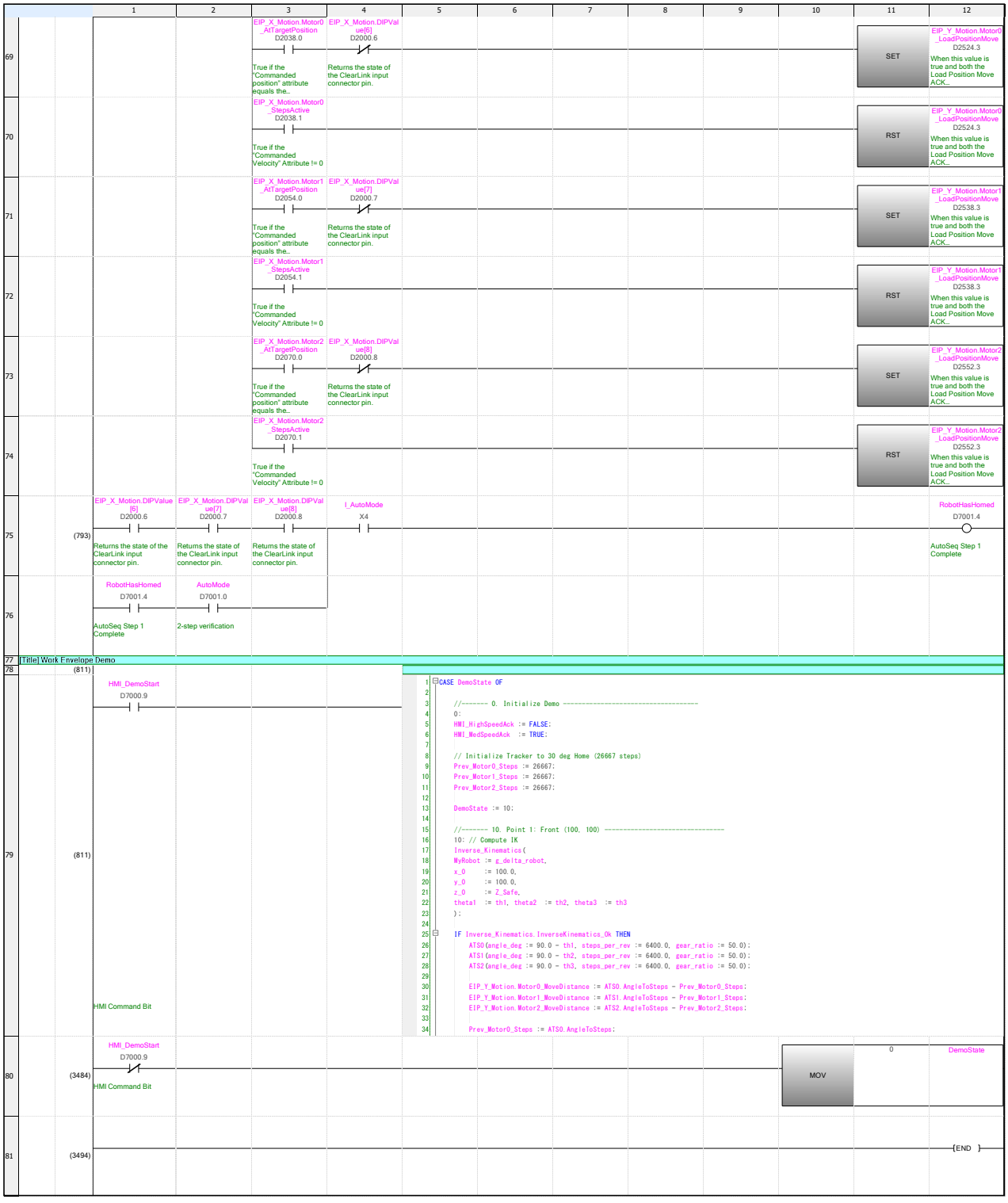
```
38                              tempReal := FrameBuffer[WriteIdx].CorrectedXPos[sortJ];
39                              FrameBuffer[WriteIdx].CorrectedXPos[sortJ] :=
   FrameBuffer[WriteIdx].CorrectedXPos[sortJ + 1];
40                              FrameBuffer[WriteIdx].CorrectedXPos[sortJ + 1] := tempReal;
41
42                              // Swap Y
43                              tempReal := FrameBuffer[WriteIdx].CorrectedYPos[sortJ];
44                              FrameBuffer[WriteIdx].CorrectedYPos[sortJ] :=
   FrameBuffer[WriteIdx].CorrectedYPos[sortJ + 1];
45                              FrameBuffer[WriteIdx].CorrectedYPos[sortJ + 1] := tempReal;
46
47                              // Swap Z
48                              tempReal := FrameBuffer[WriteIdx].CorrectedZPos[sortJ];
49                              FrameBuffer[WriteIdx].CorrectedZPos[sortJ] :=
   FrameBuffer[WriteIdx].CorrectedZPos[sortJ + 1];
50                              FrameBuffer[WriteIdx].CorrectedZPos[sortJ + 1] := tempReal;
51
52                              // Swap Class ID
53                              tempInt := FrameBuffer[WriteIdx].PartClass[sortJ];
54                              FrameBuffer[WriteIdx].PartClass[sortJ] :=
   FrameBuffer[WriteIdx].PartClass[sortJ + 1];
55                              FrameBuffer[WriteIdx].PartClass[sortJ + 1] := tempInt;
56                          END_IF;
57                      END_FOR;
58                  END_FOR;
59              END_IF;
60
61              // Advance write pointer
62              WriteIdx := (WriteIdx + 1) MOD 6;
63              IF FrameCount < 5 THEN
64                  FrameCount := FrameCount + 1;
65              END_IF;
66
67              LastVisionFrameID := VisionFrameID;
68          END_IF;
69
```


[Title] Manual Signals
These are used to prevent having to use SET'S and RESET,s in HMI. My preference. get over it
HMI_RobotState
D7000.6
HMI Command Bit
RobotEnable
Secondary Command Bit / Status Bit
RobotDisable
O_MotorPower
Y0
SM401
Always OFF
RST
HMI_MagnetPowerState
D7000.7
MagnetPowerDisable
MagnetPowerEnable
HMI_RobotHomeState
D7000.8
RobotHomeRunning
RobotHomeStopped
[Title] Signal Commands
I_ManualMode
X5
I_AutoMode
X4
O_ElectromagnetPower
Y1
TurnOnElectromagnet_Auto
D7001.3
Signal Command
RobotHomeRunning
HomeRobot_Auto
D7001.1
AutoSeq Step 1. Home Robot (Signal Command)
RobotHasHomed
D7001.4
AutoSeq Step 1 Complete
OUTH
HomeRobotTimer
K500
Time we expect robot to home in, if not trigger a fault
[Title] Fault Check
Main Error check
If any faults present, SET Vision State to ERROR
S_MotionFault
D204
Used for Motion Fault Banner
E_MachineError.3
D205.3
Used for Machine Error Display (Banner)
Latch Fault Status to prevent unwanted motion upon startup


1
2
3
4
5
6
7
8
9
10
11
12
22
(361)
IKFault
ClearMachineFaults
D200.0
S_MotionFault.0
D204.0
Used for Motion Fault Banner
23
S_MotionFault.0
D204.0
Used for Motion Fault Banner
RST
IKFault
24
(452)
I_AutoMode
X4
O_MotorPower
Y0
ClearMachineFaults
D200.0
S_MotionFault.1
D204.1
Used for Motion Fault Banner
25
S_MotionFault.1
D204.1
Used for Motion Fault Banner
26
(462)
I_AutoMode
X4
EIP_X_Motion.Motor0_Enabled
D2038.A
ClearMachineFaults
D200.0
S_MotionFault.2
D204.2
Used for Motion Fault Banner
True if the M-Connector Enable output is Asserted AND...
27
EIP_X_Motion.Motor1_Enabled
D2054.A
True if the M-Connector Enable output is Asserted AND...
28
EIP_X_Motion.Motor2_Enabled
D2070.A
True if the M-Connector Enable output is Asserted AND...
29
S_MotionFault.2
D204.2
Used for Motion Fault Banner
30
(479)
I_AutoMode
X4
EIP_X_Motion.Motor0_InFault
D2038.9
ClearMachineFaults
D200.0
S_MotionFault.3
D204.3
Used for Motion Fault Banner
True if the HLFB signal on the M-Connector is De-asserted...
31
EIP_X_Motion.Motor1_InFault
D2054.9
True if the HLFB signal on the M-Connector is De-asserted...
32
EIP_X_Motion.Motor2_InFault
D2070.9
True if the HLFB signal on the M-Connector is De-asserted...
33
S_MotionFault.3
D204.3
Used for Motion Fault Banner
34
[Title] Speed Setting
35
(496)
HMI_SlowSpeed
D7000.0
HMI_MedSpeed
D7000.1
HMI_HighSpeed
D7000.2
HMI_SlowSpeedAck
D7000.3
Signal Command
Signal Command
Signal Command
Status Signal (HMI Lamp)
36
HMI_SlowSpeedAck
D7000.3
Status Signal (HMI Lamp)
DMOV
10000
EIP_Y_Motion.Motor0_VelocityLimit
D2514
This represents the velocity limit used for the commanded positional move. Unit...
37
DMOV
10000
EIP_Y_Motion.Motor0_AccelerationLimit
D2516
This represents the acceleration limit used for the commanded position/velocity mov...


1
2
3
4
5
6
7
8
9
10
11
12
38
DMOV
10000
EIP_Y_Motion.Motor0_JogVelocity
D2520
This represents the target velocity of the next commanded velocity move. Units...
39
DMOV
10000
EIP_Y_Motion.Motor1_VelocityLimit
D2528
This represents the velocity limit used for the commanded positional move. Unit...
40
DMOV
10000
EIP_Y_Motion.Motor1_AccelerationLimit
D2530
This represents the acceleration limit used for the commanded position/velocity mov...
41
DMOV
10000
EIP_Y_Motion.Motor1_JogVelocity
D2534
This represents the target velocity of the next commanded velocity move. Units...
42
DMOV
10000
EIP_Y_Motion.Motor2_VelocityLimit
D2542
This represents the velocity limit used for the commanded positional move. Unit...
43
DMOV
10000
EIP_Y_Motion.Motor2_AccelerationLimit
D2544
This represents the acceleration limit used for the commanded position/velocity mov...
44
DMOV
10000
EIP_Y_Motion.Motor2_JogVelocity
D2548
This represents the target velocity of the next commanded velocity move. Units...
45
(583)
HMI_MedSpeed
D7000.1
Signal Command
HMI_SlowSpeed
D7000.0
Signal Command
HMI_HighSpeed
D7000.2
Signal Command
HMI_MedSpeedAck
D7000.4
Status Signal (HMI Lamp)
46
HMI_MedSpeedAck
D7000.4
Status Signal (HMI Lamp)
DMOV
10000
EIP_Y_Motion.Motor0_VelocityLimit
D2514
This represents the velocity limit used for the commanded positional move. Unit...
47
DMOV
100000
EIP_Y_Motion.Motor0_AccelerationLimit
D2516
This represents the acceleration limit used for the commanded position/velocity mov...
48
DMOV
10000
EIP_Y_Motion.Motor0_JogVelocity
D2520
This represents the target velocity of the next commanded velocity move. Units...
49
DMOV
10000
EIP_Y_Motion.Motor1_VelocityLimit
D2528
This represents the velocity limit used for the commanded positional move. Unit...
50
DMOV
100000
EIP_Y_Motion.Motor1_AccelerationLimit
D2530
This represents the acceleration limit used for the commanded position/velocity mov...
51
DMOV
10000
EIP_Y_Motion.Motor1_JogVelocity
D2534
This represents the target velocity of the next commanded velocity move. Units...
52
DMOV
10000
EIP_Y_Motion.Motor2_VelocityLimit
D2542
This represents the velocity limit used for the commanded positional move. Unit...


1 2 3 4 5 6 7 8 9 10 11 12
53 DMOV 100000 EIP_Y_Motion.Motor2_AccelerationLimit D2544 This represents the acceleration limit used for the commanded position/velocity mov...
54 DMOV 10000 EIP_Y_Motion.Motor2_JogVelocity D2548 This represents the target velocity of the next commanded velocity move. Units...
55 (643) HMI_HighSpeed D7000.2 Signal Command
HMI_MedSpeed D7000.1 Signal Command
HMI_SlowSpeed D7000.0 Signal Command
HMI_HighSpeedAck D7000.5 Status Signal (HMI Lamp)
56 HMI_HighSpeedAck D7000.5 Status Signal (HMI Lamp)
DMOV 200000 EIP_Y_Motion.Motor0_VelocityLimit D2514 This represents the velocity limit used for the commanded positional move. Unit...
57 DMOV 390000 EIP_Y_Motion.Motor0_AccelerationLimit D2516 This represents the acceleration limit used for the commanded position/velocity mov...
58 DMOV 200000 EIP_Y_Motion.Motor0_JogVelocity D2520 This represents the target velocity of the next commanded velocity move. Units...
59 DMOV 200000 EIP_Y_Motion.Motor1_VelocityLimit D2528 This represents the velocity limit used for the commanded positional move. Unit...
60 DMOV 320000 EIP_Y_Motion.Motor1_AccelerationLimit D2530 This represents the acceleration limit used for the commanded position/velocity mov...
61 DMOV 200000 EIP_Y_Motion.Motor1_JogVelocity D2534 This represents the target velocity of the next commanded velocity move. Units...
62 DMOV 200000 EIP_Y_Motion.Motor2_VelocityLimit D2542 This represents the velocity limit used for the commanded positional move. Unit...
63 DMOV 320000 EIP_Y_Motion.Motor2_AccelerationLimit D2544 This represents the acceleration limit used for the commanded position/velocity mov...
64 DMOV 200000 EIP_Y_Motion.Motor2_JogVelocity D2548 This represents the target velocity of the next commanded velocity move. Units...
65 [Title] Homing Seq
66 (703) I_AutoMode X4
HomeRobot_Auto D7001.1 AutoSeq Step 1. Home Robot (Signal Command)
DMOV 300 EIP_Y_Motion.Motor0_MoveDistance D2512 This represents the target position of the next commanded positional move.
67 DMOV 300 EIP_Y_Motion.Motor1_MoveDistance D2526 This represents the target position of the next commanded positional move.
68 DMOV 300 EIP_Y_Motion.Motor2_MoveDistance D2540 This represents the target position of the next commanded positional move.


1
2
3
4
5
6
7
8
9
10
11
12
69
EIP_X_Motion.Motor0_AtTargetPosition
D2038.0
True if the "Commanded position" attribute equals the..
EIP_X_Motion.DIPValue[6]
D2000.6
Returns the state of the ClearLink input connector pin.
SET
EIP_Y_Motion.Motor0_LoadPositionMove
D2524.3
When this value is true and both the Load Position Move ACK...
70
EIP_X_Motion.Motor0_StepsActive
D2038.1
True if the "Commanded Velocity" Attribute != 0
RST
EIP_Y_Motion.Motor0_LoadPositionMove
D2524.3
When this value is true and both the Load Position Move ACK...
71
EIP_X_Motion.Motor1_AtTargetPosition
D2054.0
True if the "Commanded position" attribute equals the..
EIP_X_Motion.DIPValue[7]
D2000.7
Returns the state of the ClearLink input connector pin.
SET
EIP_Y_Motion.Motor1_LoadPositionMove
D2538.3
When this value is true and both the Load Position Move ACK...
72
EIP_X_Motion.Motor1_StepsActive
D2054.1
True if the "Commanded Velocity" Attribute != 0
RST
EIP_Y_Motion.Motor1_LoadPositionMove
D2538.3
When this value is true and both the Load Position Move ACK...
73
EIP_X_Motion.Motor2_AtTargetPosition
D2070.0
True if the "Commanded position" attribute equals the..
EIP_X_Motion.DIPValue[8]
D2000.8
Returns the state of the ClearLink input connector pin.
SET
EIP_Y_Motion.Motor2_LoadPositionMove
D2552.3
When this value is true and both the Load Position Move ACK...
74
EIP_X_Motion.Motor2_StepsActive
D2070.1
True if the "Commanded Velocity" Attribute != 0
RST
EIP_Y_Motion.Motor2_LoadPositionMove
D2552.3
When this value is true and both the Load Position Move ACK...
75
(793)
EIP_X_Motion.DIPValue[6]
D2000.6
Returns the state of the ClearLink input connector pin.
EIP_X_Motion.DIPValue[7]
D2000.7
Returns the state of the ClearLink input connector pin.
EIP_X_Motion.DIPValue[8]
D2000.8
Returns the state of the ClearLink input connector pin.
I_AutoMode
X4
RobotHasHomed
D7001.4
AutoSeq Step 1 Complete
76
RobotHasHomed
D7001.4
AutoSeq Step 1 Complete
AutoMode
D7001.0
2-step verification
77
[Title] Work Envelope Demo
78
(811)
79
(811)
HMI_DemoStart
D7000.9
HMI Command Bit
CASE DemoState OF
//------- 0. Initialize Demo ----------
0:
HMI_HighSpeedAck := FALSE;
HMI_MedSpeedAck := TRUE;
// Initialize Tracker to 30 deg Home (26667 steps)
Prev_Motor0_Steps := 26667;
Prev_Motor1_Steps := 26667;
Prev_Motor2_Steps := 26667;
DemoState := 10;
//------- 10. Point 1: Front (100, 100) ----------
10: // Compute IK
Inverse_Kinematics(
MyRobot := g_delta_robot,
x_0 := 100.0,
y_0 := 100.0,
z_0 := Z_Safe,
theta1 := th1, theta2 := th2, theta3 := th3
);
IF Inverse_Kinematics.InverseKinematics_Ok THEN
ATS0(angle_deg := 90.0 - th1, steps_per_rev := 6400.0, gear_ratio := 50.0);
ATS1(angle_deg := 90.0 - th2, steps_per_rev := 6400.0, gear_ratio := 50.0);
ATS2(angle_deg := 90.0 - th3, steps_per_rev := 6400.0, gear_ratio := 50.0);
EIP_Y_Motion.Motor0_MoveDistance := ATS0.AngleToSteps - Prev_Motor0_Steps;
EIP_Y_Motion.Motor1_MoveDistance := ATS1.AngleToSteps - Prev_Motor1_Steps;
EIP_Y_Motion.Motor2_MoveDistance := ATS2.AngleToSteps - Prev_Motor2_Steps;
Prev_Motor0_Steps := ATS0.AngleToSteps;
80
(3484)
HMI_DemoStart
D7000.9
HMI Command Bit
MOV
0
DemoState
81
(3494)
{END }

Inline Structured Text: Motion (Row79)

```
CASE DemoState OF

    //------- 0. Initialize Demo -----------------------------------
    0:
    HMI_HighSpeedAck := FALSE;
    HMI_MedSpeedAck  := TRUE;

    // Initialize Tracker to 30 deg Home (26667 steps)
    Prev_Motor0_Steps := 26667;
    Prev_Motor1_Steps := 26667;
    Prev_Motor2_Steps := 26667;

    DemoState := 10;

    //------- 10. Point 1: Front (100, 100) ------------------------------
    10: // Compute IK
    Inverse_Kinematics(
    MyRobot := g_delta_robot,
    x_0     := 100.0,
    y_0     := 100.0,
    z_0     := Z_Safe,
    theta1  := th1, theta2  := th2, theta3  := th3
    );

    IF Inverse_Kinematics.InverseKinematics_Ok THEN
        ATS0(angle_deg := 90.0 - th1, steps_per_rev := 6400.0, gear_ratio := 50.0);
        ATS1(angle_deg := 90.0 - th2, steps_per_rev := 6400.0, gear_ratio := 50.0);
        ATS2(angle_deg := 90.0 - th3, steps_per_rev := 6400.0, gear_ratio := 50.0);

        EIP_Y_Motion.Motor0_MoveDistance := ATS0.AngleToSteps - Prev_Motor0_Steps;
        EIP_Y_Motion.Motor1_MoveDistance := ATS1.AngleToSteps - Prev_Motor1_Steps;
        EIP_Y_Motion.Motor2_MoveDistance := ATS2.AngleToSteps - Prev_Motor2_Steps;

        Prev_Motor0_Steps := ATS0.AngleToSteps;
        Prev_Motor1_Steps := ATS1.AngleToSteps;
        Prev_Motor2_Steps := ATS2.AngleToSteps;

        EIP_Y_Motion.Motor0_LoadPositionMove := TRUE;
        EIP_Y_Motion.Motor1_LoadPositionMove := TRUE;
        EIP_Y_Motion.Motor2_LoadPositionMove := TRUE;
        DemoState := 11;
    END_IF;

    11: // Wait Ack
    IF EIP_X_Motion.Motor0_LoadPositionMoveAck AND EIP_X_Motion.Motor1_LoadPositionMoveAck AND
EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
```

Inline Structured Text: Motion (Row79)

```
        EIP_Y_Motion.Motor0_LoadPositionMove := FALSE;
        EIP_Y_Motion.Motor1_LoadPositionMove := FALSE;
        EIP_Y_Motion.Motor2_LoadPositionMove := FALSE;
        DemoState := 12;
    END_IF;

    12: // Wait Ack Clear
    IF NOT EIP_X_Motion.Motor0_LoadPositionMoveAck AND NOT EIP_X_Motion.Motor1_LoadPositionMoveAck AND
NOT EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
        DemoState := 13;
    END_IF;

    13: // Wait Completion
    IF NOT EIP_X_Motion.Motor0_StepsActive AND EIP_X_Motion.Motor0_AtTargetPosition AND
        NOT EIP_X_Motion.Motor1_StepsActive AND EIP_X_Motion.Motor1_AtTargetPosition AND
        NOT EIP_X_Motion.Motor2_StepsActive AND EIP_X_Motion.Motor2_AtTargetPosition THEN
        DemoState := 20;
    END_IF;

    //------- 20. Point 2: Right (-100,100) --------------------------------
    20: // Compute IK
    Inverse_Kinematics(
    MyRobot := g_delta_robot,
    x_0     := -100.0,
    y_0     := 100.0,
    z_0     := Z_Safe,
    theta1  := th1, theta2  := th2, theta3  := th3
    );

    IF Inverse_Kinematics.InverseKinematics_Ok THEN
        ATS0(angle_deg := 90.0 - th1, steps_per_rev := 6400.0, gear_ratio := 50.0);
        ATS1(angle_deg := 90.0 - th2, steps_per_rev := 6400.0, gear_ratio := 50.0);
        ATS2(angle_deg := 90.0 - th3, steps_per_rev := 6400.0, gear_ratio := 50.0);

        EIP_Y_Motion.Motor0_MoveDistance := ATS0.AngleToSteps - Prev_Motor0_Steps;
        EIP_Y_Motion.Motor1_MoveDistance := ATS1.AngleToSteps - Prev_Motor1_Steps;
        EIP_Y_Motion.Motor2_MoveDistance := ATS2.AngleToSteps - Prev_Motor2_Steps;

        Prev_Motor0_Steps := ATS0.AngleToSteps;
        Prev_Motor1_Steps := ATS1.AngleToSteps;
        Prev_Motor2_Steps := ATS2.AngleToSteps;

        EIP_Y_Motion.Motor0_LoadPositionMove := TRUE;
        EIP_Y_Motion.Motor1_LoadPositionMove := TRUE;
        EIP_Y_Motion.Motor2_LoadPositionMove := TRUE;
        DemoState := 21;
```

Inline Structured Text: Motion (Row79)

```
    END_IF;

    21: // Wait Ack
    IF EIP_X_Motion.Motor0_LoadPositionMoveAck AND EIP_X_Motion.Motor1_LoadPositionMoveAck AND EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
        EIP_Y_Motion.Motor0_LoadPositionMove := FALSE;
        EIP_Y_Motion.Motor1_LoadPositionMove := FALSE;
        EIP_Y_Motion.Motor2_LoadPositionMove := FALSE;
        DemoState := 22;
    END_IF;

    22: // Wait Ack Clear
    IF NOT EIP_X_Motion.Motor0_LoadPositionMoveAck AND NOT EIP_X_Motion.Motor1_LoadPositionMoveAck AND NOT EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
        DemoState := 23;
    END_IF;

    23: // Wait Completion
    IF NOT EIP_X_Motion.Motor0_StepsActive AND EIP_X_Motion.Motor0_AtTargetPosition AND
        NOT EIP_X_Motion.Motor1_StepsActive AND EIP_X_Motion.Motor1_AtTargetPosition AND
        NOT EIP_X_Motion.Motor2_StepsActive AND EIP_X_Motion.Motor2_AtTargetPosition THEN
        DemoState := 30;
    END_IF;

    //------- 30. Point 3: Back (-100, -100) ---------------------------------
    30: // Compute IK
    Inverse_Kinematics(
    MyRobot := g_delta_robot,
    x_0     := -100.0,
    y_0     := -100.0,
    z_0     := Z_Safe,
    theta1  := th1, theta2  := th2, theta3  := th3
    );

    IF Inverse_Kinematics.InverseKinematics_Ok THEN
        ATS0(angle_deg := 90.0 - th1, steps_per_rev := 6400.0, gear_ratio := 50.0);
        ATS1(angle_deg := 90.0 - th2, steps_per_rev := 6400.0, gear_ratio := 50.0);
        ATS2(angle_deg := 90.0 - th3, steps_per_rev := 6400.0, gear_ratio := 50.0);

        EIP_Y_Motion.Motor0_MoveDistance := ATS0.AngleToSteps - Prev_Motor0_Steps;
        EIP_Y_Motion.Motor1_MoveDistance := ATS1.AngleToSteps - Prev_Motor1_Steps;
        EIP_Y_Motion.Motor2_MoveDistance := ATS2.AngleToSteps - Prev_Motor2_Steps;

        Prev_Motor0_Steps := ATS0.AngleToSteps;
        Prev_Motor1_Steps := ATS1.AngleToSteps;
        Prev_Motor2_Steps := ATS2.AngleToSteps;
```

Inline Structured Text: Motion (Row79)

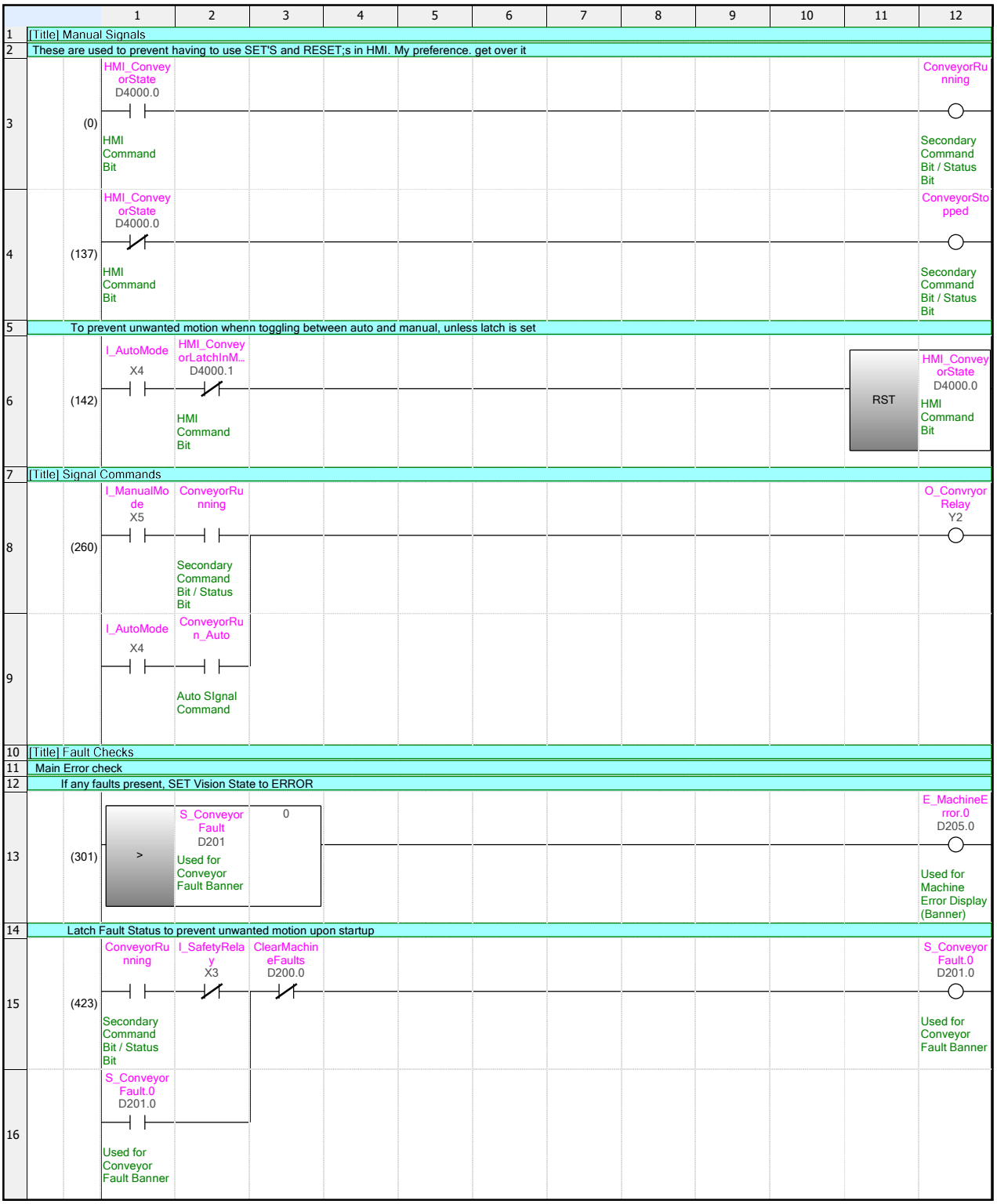
```
135
136         EIP_Y_Motion.Motor0_LoadPositionMove := TRUE;
137         EIP_Y_Motion.Motor1_LoadPositionMove := TRUE;
138         EIP_Y_Motion.Motor2_LoadPositionMove := TRUE;
139         DemoState := 31;
140     END_IF;
141
142     31: // Wait Ack
143     IF EIP_X_Motion.Motor0_LoadPositionMoveAck AND EIP_X_Motion.Motor1_LoadPositionMoveAck AND
    EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
144         EIP_Y_Motion.Motor0_LoadPositionMove := FALSE;
145         EIP_Y_Motion.Motor1_LoadPositionMove := FALSE;
146         EIP_Y_Motion.Motor2_LoadPositionMove := FALSE;
147         DemoState := 32;
148     END_IF;
149
150     32: // Wait Ack Clear
151     IF NOT EIP_X_Motion.Motor0_LoadPositionMoveAck AND NOT EIP_X_Motion.Motor1_LoadPositionMoveAck AND
    NOT EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
152         DemoState := 33;
153     END_IF;
154
155     33: // Wait Completion
156     IF NOT EIP_X_Motion.Motor0_StepsActive AND EIP_X_Motion.Motor0_AtTargetPosition AND
157         NOT EIP_X_Motion.Motor1_StepsActive AND EIP_X_Motion.Motor1_AtTargetPosition AND
158         NOT EIP_X_Motion.Motor2_StepsActive AND EIP_X_Motion.Motor2_AtTargetPosition THEN
159         DemoState := 40;
160     END_IF;
161
162     //------- 40. Point 4: Left (-100, 0) --------------------------------
163     40: // Compute IK
164     Inverse_Kinematics(
165     MyRobot := g_delta_robot,
166     x_0     := 100.0,
167     y_0     := -100.0,
168     z_0     := Z_Safe,
169     theta1  := th1, theta2  := th2, theta3  := th3
170     );
171
172     IF Inverse_Kinematics.InverseKinematics_Ok THEN
173         ATS0(angle_deg := 90.0 - th1, steps_per_rev := 6400.0, gear_ratio := 50.0);
174         ATS1(angle_deg := 90.0 - th2, steps_per_rev := 6400.0, gear_ratio := 50.0);
175         ATS2(angle_deg := 90.0 - th3, steps_per_rev := 6400.0, gear_ratio := 50.0);
176
177         EIP_Y_Motion.Motor0_MoveDistance := ATS0.AngleToSteps - Prev_Motor0_Steps;
178         EIP_Y_Motion.Motor1_MoveDistance := ATS1.AngleToSteps - Prev_Motor1_Steps;
```

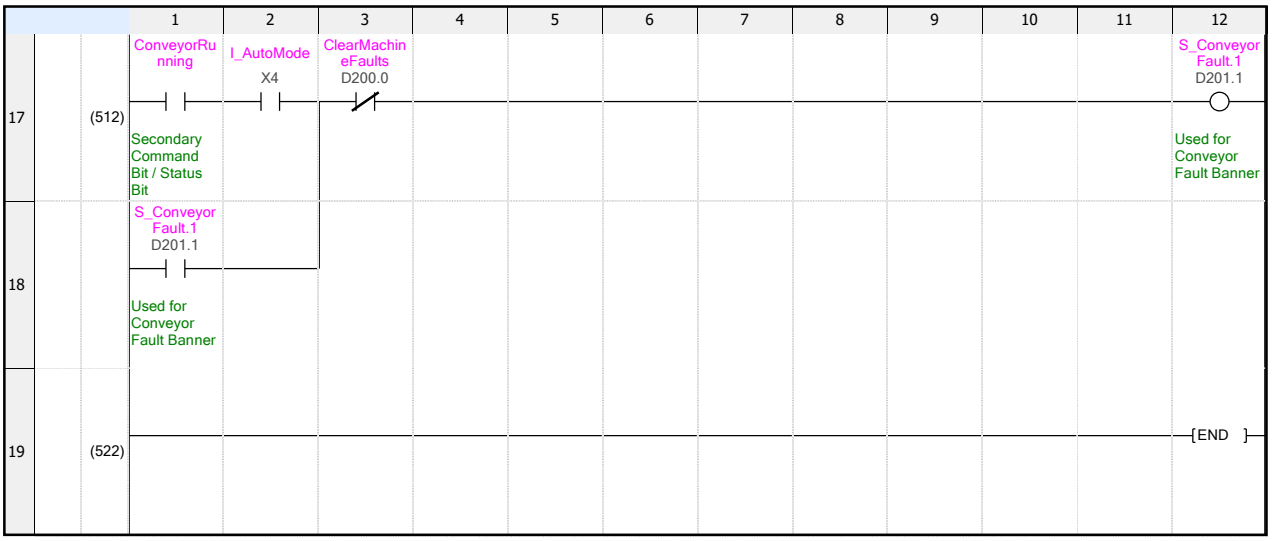
Inline Structured Text: Motion (Row79)

```
179         EIP_Y_Motion.Motor2_MoveDistance := ATS2.AngleToSteps - Prev_Motor2_Steps;
180
181         Prev_Motor0_Steps := ATS0.AngleToSteps;
182         Prev_Motor1_Steps := ATS1.AngleToSteps;
183         Prev_Motor2_Steps := ATS2.AngleToSteps;
184
185         EIP_Y_Motion.Motor0_LoadPositionMove := TRUE;
186         EIP_Y_Motion.Motor1_LoadPositionMove := TRUE;
187         EIP_Y_Motion.Motor2_LoadPositionMove := TRUE;
188         DemoState := 41;
189     END_IF;
190
191     41: // Wait Ack
192     IF EIP_X_Motion.Motor0_LoadPositionMoveAck AND EIP_X_Motion.Motor1_LoadPositionMoveAck AND
    EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
193         EIP_Y_Motion.Motor0_LoadPositionMove := FALSE;
194         EIP_Y_Motion.Motor1_LoadPositionMove := FALSE;
195         EIP_Y_Motion.Motor2_LoadPositionMove := FALSE;
196         DemoState := 42;
197     END_IF;
198
199     42: // Wait Ack Clear
200     IF NOT EIP_X_Motion.Motor0_LoadPositionMoveAck AND NOT EIP_X_Motion.Motor1_LoadPositionMoveAck AND
    NOT EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
201         DemoState := 43;
202     END_IF;
203
204     43: // Wait Completion
205     IF NOT EIP_X_Motion.Motor0_StepsActive AND EIP_X_Motion.Motor0_AtTargetPosition AND
206         NOT EIP_X_Motion.Motor1_StepsActive AND EIP_X_Motion.Motor1_AtTargetPosition AND
207         NOT EIP_X_Motion.Motor2_StepsActive AND EIP_X_Motion.Motor2_AtTargetPosition THEN
208         DemoState := 50;
209     END_IF;
210
211     //------- 50. Point 5: Center (0, 0) --------------------------------
212     50: // Compute IK
213     Inverse_Kinematics(
214     MyRobot := g_delta_robot,
215     x_0     := 0.0,
216     y_0     := 0.0,
217     z_0     := Z_Safe,
218     theta1  := th1, theta2  := th2, theta3  := th3
219     );
220
221     IF Inverse_Kinematics.InverseKinematics_Ok THEN
222         ATS0(angle_deg := 90.0 - th1, steps_per_rev := 6400.0, gear_ratio := 50.0);
```

Inline Structured Text: Motion (Row79)

```
223         ATS1(angle_deg := 90.0 - th2, steps_per_rev := 6400.0, gear_ratio := 50.0);
224         ATS2(angle_deg := 90.0 - th3, steps_per_rev := 6400.0, gear_ratio := 50.0);
225
226         EIP_Y_Motion.Motor0_MoveDistance := ATS0.AngleToSteps - Prev_Motor0_Steps;
227         EIP_Y_Motion.Motor1_MoveDistance := ATS1.AngleToSteps - Prev_Motor1_Steps;
228         EIP_Y_Motion.Motor2_MoveDistance := ATS2.AngleToSteps - Prev_Motor2_Steps;
229
230         Prev_Motor0_Steps := ATS0.AngleToSteps;
231         Prev_Motor1_Steps := ATS1.AngleToSteps;
232         Prev_Motor2_Steps := ATS2.AngleToSteps;
233
234         EIP_Y_Motion.Motor0_LoadPositionMove := TRUE;
235         EIP_Y_Motion.Motor1_LoadPositionMove := TRUE;
236         EIP_Y_Motion.Motor2_LoadPositionMove := TRUE;
237         DemoState := 51;
238     END_IF;
239
240     51: // Wait Ack
241     IF EIP_X_Motion.Motor0_LoadPositionMoveAck AND EIP_X_Motion.Motor1_LoadPositionMoveAck AND
    EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
242         EIP_Y_Motion.Motor0_LoadPositionMove := FALSE;
243         EIP_Y_Motion.Motor1_LoadPositionMove := FALSE;
244         EIP_Y_Motion.Motor2_LoadPositionMove := FALSE;
245         DemoState := 52;
246     END_IF;
247
248     52: // Wait Ack Clear
249     IF NOT EIP_X_Motion.Motor0_LoadPositionMoveAck AND NOT EIP_X_Motion.Motor1_LoadPositionMoveAck AND
    NOT EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
250         DemoState := 53;
251     END_IF;
252
253     53: // Wait Completion
254     IF NOT EIP_X_Motion.Motor0_StepsActive AND EIP_X_Motion.Motor0_AtTargetPosition AND
255         NOT EIP_X_Motion.Motor1_StepsActive AND EIP_X_Motion.Motor1_AtTargetPosition AND
256         NOT EIP_X_Motion.Motor2_StepsActive AND EIP_X_Motion.Motor2_AtTargetPosition THEN
257         DemoState := 10; // Loop back to Point 1
258     END_IF;
259
260 END_CASE;
261
```


1 2 3 4 5 6 7 8 9 10 11 12
1 [Title] Manual Signals
2 These are used to prevent having to use SET'S and RESET;s in HMI. My preference. get over it
3 (0) HMI_ConveyorState D4000.0 HMI Command Bit
ConveyorRunning Secondary Command Bit / Status Bit
4 (137) HMI_ConveyorState D4000.0 HMI Command Bit
ConveyorStopped Secondary Command Bit / Status Bit
5 To prevent unwanted motion whenn toggling between auto and manual, unless latch is set
6 (142) I_AutoMode X4
HMI_ConveyorLatchInM... D4000.1 HMI Command Bit
RST HMI_ConveyorState D4000.0 HMI Command Bit
7 [Title] Signal Commands
8 (260) I_ManualMode X5
ConveyorRunning Secondary Command Bit / Status Bit
O_Convryor Relay Y2
9 I_AutoMode X4
ConveyorRun_Auto Auto SIgnal Command
10 [Title] Fault Checks
11 Main Error check
12 If any faults present, SET Vision State to ERROR
13 (301) > S_Conveyor Fault D201 Used for Conveyor Fault Banner
0
E_MachineError.0 D205.0 Used for Machine Error Display (Banner)
14 Latch Fault Status to prevent unwanted motion upon startup
15 (423) ConveyorRunning Secondary Command Bit / Status Bit
I_SafetyRelay X3
ClearMachineFaults D200.0
S_Conveyor Fault.0 D201.0 Used for Conveyor Fault Banner
16 S_Conveyor Fault.0 D201.0 Used for Conveyor Fault Banner


1
2
3
4
5
6
7
8
9
10
11
12
17
(512)
ConveyorRunning
I_AutoMode
X4
ClearMachineFaults
D200.0
S_ConveyorFault.1
D201.1
Secondary Command Bit / Status Bit
Used for Conveyor Fault Banner
18
S_ConveyorFault.1
D201.1
Used for Conveyor Fault Banner
19
(522)
END

```
IF I_AutoMode AND I_SafetyRelay AND (E_MachineError = 0) THEN
    AutoMode := TRUE;

    //---------- System Timer ----------------------------------
    IF HMI_BypassConveyor THEN
        OUTH(NOT VisionScanTimer.S, VisionScanTimer, ScanInterval);
        ELSE
        OUTH(ConveyorRun_Auto AND NOT VisionScanTimer.S, VisionScanTimer,
ScanInterval);
    END_IF;

    IF VisionScanTimer.S THEN
        AutoTriggerCamera := TRUE;
        StartJobTimer := TRUE;
        VisionScanTimer.N := 0;
        VisionScanTimer.S := 0;
    END_IF;
    OUTH(StartJobTimer, VisionJobTimeCount, VisionExpectedJobTime);
    IF VisionJobTimeCount.S THEN
        StartJobTimer := FALSE;
        VisionFrameID := VisionFrameID + 1;
        VisionJobTimeCount.N := 0;
        VisionJobTimeCount.S := 0;
    END_IF;

    CASE RobotState OF

        0:
        RobotState := 1;

        //------- 1. Home Robot
-------------------------------------------------
        1:
        HomeRobot_Auto := TRUE;
        HMI_HighSpeedAck := FALSE;
        HMI_SlowSpeedAck := TRUE;
        AutoMode := TRUE;
        Homing_TurnOffElectromagnet := TRUE;
        RobotState := 2;

        //------- 2. Sync Tracker -------------------------------
        2:
        IF RobotHasHomed THEN
            HomeRobot_Auto := FALSE;
            HMI_HighSpeedAck := TRUE;
            HMI_SlowSpeedAck := FALSE;
            IF HMI_BypassConveyor THEN
```

```
46                  ConveyorRun_Auto := FALSE;
47                  ELSE
48                  ConveyorRun_Auto := TRUE;
49              END_IF;
50
51              // Initialize Tracker to 30 deg Home (26667 steps)
52              Prev_Motor0_Steps := 26667;
53              Prev_Motor1_Steps := 26667;
54              Prev_Motor2_Steps := 26667;
55
56              RobotState := 10;   // Main Loop (Idle)
57          END_IF;
58
59          //------------- 10. Main Loop (Idle) ---------------------------------------------------
60          10:
61          IF FrameCount > 4 THEN
62              ActiveFrame := FrameBuffer[ReadIdx];
63              ReadIdx := (ReadIdx + 1) MOD 6;
64              FrameCount := FrameCount - 1;
65
66              loopIndex := 0;
67              PartsTotal := ActiveFrame.PartsTotal;
68
69              RobotState := 30;   // Step-Thru Main Loop
70          END_IF;
71
72          //-------- 30. Step-Thru Main Loop ----------------------------------------
73          30:
74          IF loopIndex < PartsTotal THEN
75              RobotState := 40;   // Compute IK for "Safe Pick"
76              ELSE
77              RobotState := 10;   // Main Loop (Idle)
78          END_IF;
79
80          //------- 40. Compute IK for "Safe Pick" ---------------------------------
81          40:
82
83          //IF loopIndex > 0 THEN
84              // Total Time = ((Full Intervals Passed) * ScanInterval) + Current Cycle Time
85              CurrentTCP.X_Position := ActiveFrame.CorrectedXPos[loopIndex] + (loopIndex * 20) + (BeltSpeed_mm_s * VisionScanTimer.N / 100);//(((VisionFrameID - 3) - ActiveFrame.FrameID) * ScanInterval + VisionScanTimer.N) / 100);
```

```
            //ELSE
            //CurrentTCP.X_Position := ActiveFrame.CorrectedXPos[loopIndex];
        //END_IF;

        Inverse_Kinematics(
        MyRobot := g_delta_robot,
        x_0     := CurrentTCP.X_Position,
        y_0     := ActiveFrame.CorrectedYPos[loopIndex],
        z_0     := Z_Safe,
        theta1  := th1, theta2  := th2, theta3  := th3
        );

        IF Inverse_Kinematics.InverseKinematics_Ok THEN
            ATS0(angle_deg := 90.0 - th1, steps_per_rev := 6400.0, gear_ratio := 50.0);
            ATS1(angle_deg := 90.0 - th2, steps_per_rev := 6400.0, gear_ratio := 50.0);
            ATS2(angle_deg := 90.0 - th3, steps_per_rev := 6400.0, gear_ratio := 50.0);

            Motor0_Steps := ATS0.AngleToSteps - Prev_Motor0_Steps;
            Motor1_Steps := ATS1.AngleToSteps - Prev_Motor1_Steps;
            Motor2_Steps := ATS2.AngleToSteps - Prev_Motor2_Steps;

            Prev_Motor0_Steps := ATS0.AngleToSteps;
            Prev_Motor1_Steps := ATS1.AngleToSteps;
            Prev_Motor2_Steps := ATS2.AngleToSteps;

            IF ActiveFrame.CorrectedYPos[loopIndex] > 115 OR ActiveFrame.CorrectedXPos[loopIndex] > 115 THEN
                IKFault := TRUE;    // Error
                ELSE
                RobotState := 50;   // Move to "Safe Pick"
            END_IF;

            ELSE
            RobotState := 300;  // Error
        END_IF;

        //------- 50. Move to "Safe Pick" -------------------------------
        50: // Trigger Move (Ramp Start)
        StartPos0 := EIP_X_Motion.Motor0_CommandedPosition;
        StartPos1 := EIP_X_Motion.Motor1_CommandedPosition;
        StartPos2 := EIP_X_Motion.Motor2_CommandedPosition;

        EIP_Y_Motion.Motor0_MoveDistance := Motor0_Steps;
        EIP_Y_Motion.Motor1_MoveDistance := Motor1_Steps;
```

```
129         EIP_Y_Motion.Motor2_MoveDistance := Motor2_Steps;
130         EIP_Y_Motion.Motor0_LoadPositionMove := TRUE;
131         EIP_Y_Motion.Motor1_LoadPositionMove := TRUE;
132         EIP_Y_Motion.Motor2_LoadPositionMove := TRUE;
133         RobotState := 51;
134
135         51: // Wait Ack
136         IF EIP_X_Motion.Motor0_LoadPositionMoveAck AND
    EIP_X_Motion.Motor1_LoadPositionMoveAck AND
    EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
137             EIP_Y_Motion.Motor0_LoadPositionMove := FALSE;
138             EIP_Y_Motion.Motor1_LoadPositionMove := FALSE;
139             EIP_Y_Motion.Motor2_LoadPositionMove := FALSE;
140             RobotState := 52;
141         END_IF;
142
143         52: // Wait Ack Clear
144         IF NOT EIP_X_Motion.Motor0_LoadPositionMoveAck AND NOT
    EIP_X_Motion.Motor1_LoadPositionMoveAck AND NOT
    EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
145             RobotState := 53;
146         END_IF;
147
148         53: // Ramp Monitor (Blend to Pick)
149         DistTotal := ABS(Motor0_Steps) + ABS(Motor1_Steps) +
    ABS(Motor2_Steps);
150         DistDone  := ABS(EIP_X_Motion.Motor0_CommandedPosition - StartPos0) +
151         ABS(EIP_X_Motion.Motor1_CommandedPosition - StartPos1) +
152         ABS(EIP_X_Motion.Motor2_CommandedPosition - StartPos2);
153
154         IF (DistTotal > 0 AND (DistDone * 100 / DistTotal) >= RampPercent)
    OR DistTotal = 0 THEN
155             RobotState := 60;   // Compute IK for "Actual Pick"
156         END_IF;
157
158         //------ 60. Compute IK for "Actual Pick"
    -------------------------------
159         60:
160
161         //IF loopIndex > 0 THEN
162             CurrentTCP.X_Position := ActiveFrame.CorrectedXPos[loopIndex] +
    (loopIndex * 5) + (BeltSpeed_mm_s * VisionScanTimer.N /
    100);////((VisionFrameID - 3) - ActiveFrame.FrameID) * ScanInterval +
    VisionScanTimer.N) / 100);
163             //ELSE
164             //CurrentTCP.X_Position := ActiveFrame.CorrectedXPos[loopIndex];
165         //END_IF;
```

```
166
167         Inverse_Kinematics(
168         MyRobot := g_delta_robot,
169         x_0     := CurrentTCP.X_Position,
170         y_0     := ActiveFrame.CorrectedYPos[loopIndex],
171         z_0     := Z_Pick,
172         theta1  := th1, theta2  := th2, theta3  := th3
173         );
174
175         IF Inverse_Kinematics.InverseKinematics_Ok THEN
176             ATS0(angle_deg := 90.0 - th1, steps_per_rev := 6400.0,
    gear_ratio := 50.0);
177             ATS1(angle_deg := 90.0 - th2, steps_per_rev := 6400.0,
    gear_ratio := 50.0);
178             ATS2(angle_deg := 90.0 - th3, steps_per_rev := 6400.0,
    gear_ratio := 50.0);
179
180             Motor0_Steps := ATS0.AngleToSteps - Prev_Motor0_Steps;
181             Motor1_Steps := ATS1.AngleToSteps - Prev_Motor1_Steps;
182             Motor2_Steps := ATS2.AngleToSteps - Prev_Motor2_Steps;
183
184             Prev_Motor0_Steps := ATS0.AngleToSteps;
185             Prev_Motor1_Steps := ATS1.AngleToSteps;
186             Prev_Motor2_Steps := ATS2.AngleToSteps;
187
188             RobotState := 70;   // Move to "Safe Pick"
189
190             ELSE
191             RobotState := 300;
192         END_IF;
193
194         //------- 70. Move to "Actual Pick" ---------------------------------
195         70: // Trigger Move
196         EIP_Y_Motion.Motor0_MoveDistance := Motor0_Steps;
197         EIP_Y_Motion.Motor1_MoveDistance := Motor1_Steps;
198         EIP_Y_Motion.Motor2_MoveDistance := Motor2_Steps;
199         EIP_Y_Motion.Motor0_LoadPositionMove := TRUE;
200         EIP_Y_Motion.Motor1_LoadPositionMove := TRUE;
201         EIP_Y_Motion.Motor2_LoadPositionMove := TRUE;
202         RobotState := 71;
203
204         71: // Wait Ack
205         IF EIP_X_Motion.Motor0_LoadPositionMoveAck AND
    EIP_X_Motion.Motor1_LoadPositionMoveAck AND
    EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
206             EIP_Y_Motion.Motor0_LoadPositionMove := FALSE;
207             EIP_Y_Motion.Motor1_LoadPositionMove := FALSE;
```

```
208             EIP_Y_Motion.Motor2_LoadPositionMove := FALSE;
209             RobotState := 72;
210         END_IF;
211
212         72: // Wait Ack Clear
213         IF NOT EIP_X_Motion.Motor0_LoadPositionMoveAck AND NOT
    EIP_X_Motion.Motor1_LoadPositionMoveAck AND NOT
    EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
214             RobotState := 73;
215         END_IF;
216
217         73: // Wait Completion (Full Stop for Pick)
218         IF NOT EIP_X_Motion.Motor0_StepsActive AND
    EIP_X_Motion.Motor0_AtTargetPosition AND
    EIP_X_Motion.Motor1_AtTargetPosition AND
    EIP_X_Motion.Motor2_AtTargetPosition THEN
219             RobotState := 80;   // Pick Part
220         END_IF;
221
222         //------- 80. Pick Part
    -------------------------------------------
223         80:
224         PickComplete := TRUE;
225         TurnOnElectromagnet_Auto := TRUE;
226         RobotState := 90;   // Compute IK for "Safe Lift"
227
228         //------- 90. Compute IK for "Safe Lift"
    -------------------------------
229         90:
230
231         //IF loopIndex > 0 THEN
232         //  CurrentTCP.X_Position := ActiveFrame.CorrectedXPos[loopIndex] +
    (BeltSpeed_mm_s * (((VisionFrameID - 4) - ActiveFrame.FrameID) *
    ScanInterval + VisionScanTimer.N) / 100);
233         //  ELSE
234         //  CurrentTCP.X_Position := ActiveFrame.CorrectedXPos[loopIndex];
235         //END_IF;
236
237         Inverse_Kinematics(
238         MyRobot := g_delta_robot,
239         x_0     := CurrentTCP.X_Position,
240         y_0     := ActiveFrame.CorrectedYPos[loopIndex],
241         z_0     := Z_Safe,
242         theta1  := th1, theta2  := th2, theta3  := th3
243         );
244
245         IF Inverse_Kinematics.InverseKinematics_Ok THEN
```

```
246             ATS0(angle_deg := 90.0 - th1, steps_per_rev := 6400.0,
    gear_ratio := 50.0);
247             ATS1(angle_deg := 90.0 - th2, steps_per_rev := 6400.0,
    gear_ratio := 50.0);
248             ATS2(angle_deg := 90.0 - th3, steps_per_rev := 6400.0,
    gear_ratio := 50.0);
249
250             Motor0_Steps := ATS0.AngleToSteps - Prev_Motor0_Steps;
251             Motor1_Steps := ATS1.AngleToSteps - Prev_Motor1_Steps;
252             Motor2_Steps := ATS2.AngleToSteps - Prev_Motor2_Steps;
253
254             Prev_Motor0_Steps := ATS0.AngleToSteps;
255             Prev_Motor1_Steps := ATS1.AngleToSteps;
256             Prev_Motor2_Steps := ATS2.AngleToSteps;
257
258             RobotState := 100;  // Move to "Safe Lift"
259             ELSE
260             RobotState := 300;
261         END_IF;
262
263         //------- 100. Move to "Safe Lift" --------------------------------
264         100: // Trigger Move (Ramp Start)
265         StartPos0 := EIP_X_Motion.Motor0_CommandedPosition;
266         StartPos1 := EIP_X_Motion.Motor1_CommandedPosition;
267         StartPos2 := EIP_X_Motion.Motor2_CommandedPosition;
268
269         EIP_Y_Motion.Motor0_MoveDistance := Motor0_Steps;
270         EIP_Y_Motion.Motor1_MoveDistance := Motor1_Steps;
271         EIP_Y_Motion.Motor2_MoveDistance := Motor2_Steps;
272         EIP_Y_Motion.Motor0_LoadPositionMove := TRUE;
273         EIP_Y_Motion.Motor1_LoadPositionMove := TRUE;
274         EIP_Y_Motion.Motor2_LoadPositionMove := TRUE;
275         RobotState := 101;
276
277         101: // Wait Ack
278         IF EIP_X_Motion.Motor0_LoadPositionMoveAck AND
    EIP_X_Motion.Motor1_LoadPositionMoveAck AND
    EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
279             EIP_Y_Motion.Motor0_LoadPositionMove := FALSE;
280             EIP_Y_Motion.Motor1_LoadPositionMove := FALSE;
281             EIP_Y_Motion.Motor2_LoadPositionMove := FALSE;
282             RobotState := 102;
283         END_IF;
284
285         102: // Wait Ack Clear
286         IF NOT EIP_X_Motion.Motor0_LoadPositionMoveAck AND NOT
    EIP_X_Motion.Motor1_LoadPositionMoveAck AND NOT
```

```
    EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
287             RobotState := 103;
288         END_IF;
289
290         103: // Ramp Monitor (Blend to Intermediate)
291         DistTotal := ABS(Motor0_Steps) + ABS(Motor1_Steps) +
    ABS(Motor2_Steps);
292         DistDone  := ABS(EIP_X_Motion.Motor0_CommandedPosition - StartPos0) +
293         ABS(EIP_X_Motion.Motor1_CommandedPosition - StartPos1) +
294         ABS(EIP_X_Motion.Motor2_CommandedPosition - StartPos2);
295
296         IF (DistTotal > 0 AND (DistDone * 100 / DistTotal) >= RampPercent)
    OR DistTotal = 0 THEN
297             RobotState := 110;  // Compute IK for Intermediate Waypoint
298         END_IF;
299
300         //------- 110. Compute IK for Intermediate Waypoint
    ---------------------------
301         110:
302
303         //IF loopIndex > 0 THEN
304         //  CurrentTCP.X_Position := ActiveFrame.CorrectedXPos[loopIndex] +
    (BeltSpeed_mm_s * (((VisionFrameID - 4) - ActiveFrame.FrameID) *
    ScanInterval + VisionScanTimer.N) / 100);
305         //  ELSE
306         //  CurrentTCP.X_Position := ActiveFrame.CorrectedXPos[loopIndex];
307         //END_IF;
308
309             IF ActiveFrame.PartClass[loopIndex] = 1 THEN TargetX := 30.0;
    ELSE TargetX := -30.0; END_IF;
310
311             Inverse_Kinematics(
312             MyRobot := g_delta_robot,
313             x_0     := (CurrentTCP.X_Position + TargetX) / 2.0,
314             y_0     := (ActiveFrame.CorrectedYPos[loopIndex] - 120.0) / 2.0,
315             z_0     := Z_Safe,
316             theta1  := th1, theta2  := th2, theta3  := th3
317             );
318
319         IF Inverse_Kinematics.InverseKinematics_Ok THEN
320             ATS0(angle_deg := 90.0 - th1, steps_per_rev := 6400.0,
    gear_ratio := 50.0);
321             ATS1(angle_deg := 90.0 - th2, steps_per_rev := 6400.0,
    gear_ratio := 50.0);
322             ATS2(angle_deg := 90.0 - th3, steps_per_rev := 6400.0,
    gear_ratio := 50.0);
323
```

```
324             Motor0_Steps := ATS0.AngleToSteps - Prev_Motor0_Steps;
325             Motor1_Steps := ATS1.AngleToSteps - Prev_Motor1_Steps;
326             Motor2_Steps := ATS2.AngleToSteps - Prev_Motor2_Steps;
327
328             Prev_Motor0_Steps := ATS0.AngleToSteps;
329             Prev_Motor1_Steps := ATS1.AngleToSteps;
330             Prev_Motor2_Steps := ATS2.AngleToSteps;
331
332             RobotState := 120;  // Move to Intermediate Waypoint
333             ELSE
334             RobotState := 300;
335         END_IF;
336
337         //------- 120. Move to Intermediate Waypoint
    -------------------------------
338         120: // Trigger Move (Ramp Start)
339         StartPos0 := EIP_X_Motion.Motor0_CommandedPosition;
340         StartPos1 := EIP_X_Motion.Motor1_CommandedPosition;
341         StartPos2 := EIP_X_Motion.Motor2_CommandedPosition;
342
343         EIP_Y_Motion.Motor0_MoveDistance := Motor0_Steps;
344         EIP_Y_Motion.Motor1_MoveDistance := Motor1_Steps;
345         EIP_Y_Motion.Motor2_MoveDistance := Motor2_Steps;
346         EIP_Y_Motion.Motor0_LoadPositionMove := TRUE;
347         EIP_Y_Motion.Motor1_LoadPositionMove := TRUE;
348         EIP_Y_Motion.Motor2_LoadPositionMove := TRUE;
349         RobotState := 121;
350
351         121: // Wait Ack
352         IF EIP_X_Motion.Motor0_LoadPositionMoveAck AND
    EIP_X_Motion.Motor1_LoadPositionMoveAck AND
    EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
353             EIP_Y_Motion.Motor0_LoadPositionMove := FALSE;
354             EIP_Y_Motion.Motor1_LoadPositionMove := FALSE;
355             EIP_Y_Motion.Motor2_LoadPositionMove := FALSE;
356             RobotState := 122;
357         END_IF;
358
359         122: // Wait Ack Clear
360         IF NOT EIP_X_Motion.Motor0_LoadPositionMoveAck AND NOT
    EIP_X_Motion.Motor1_LoadPositionMoveAck AND NOT
    EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
361             RobotState := 123;
362         END_IF;
363
364         123: // Ramp Monitor (Blend to Safe Drop)
365         DistTotal := ABS(Motor0_Steps) + ABS(Motor1_Steps) +
```

```
    ABS(Motor2_Steps);
366         DistDone  := ABS(EIP_X_Motion.Motor0_CommandedPosition - StartPos0) +
367         ABS(EIP_X_Motion.Motor1_CommandedPosition - StartPos1) +
368         ABS(EIP_X_Motion.Motor2_CommandedPosition - StartPos2);
369
370         IF (DistTotal > 0 AND (DistDone * 100 / DistTotal) >= RampPercent)
    OR DistTotal = 0 THEN
371             RobotState := 130;  // Compute IK for Safe DropOff
372         END_IF;
373
374         //------- 130. Compute IK for Safe DropOff
    --------------------------------
375         130:
376         PickComplete := FALSE;
377
378         IF ActiveFrame.PartClass[loopIndex] = 1 THEN    // Scrap
379             Inverse_Kinematics(
380             MyRobot := g_delta_robot,
381             x_0     := 50.0,
382             y_0     := -120.0,
383             z_0     := Z_Safe,
384             theta1  := th1, theta2  := th2, theta3  := th3
385             );
386             ELSIF ActiveFrame.PartClass[loopIndex] = 2 THEN // Rework
387             Inverse_Kinematics(
388             MyRobot := g_delta_robot,
389             x_0     := -50.0,
390             y_0     := -120.0,
391             z_0     := Z_Safe,
392             theta1  := th1, theta2  := th2, theta3  := th3
393             );
394             ELSE
395             RobotState := 300; // Error: Unknown Class
396         END_IF;
397
398         IF Inverse_Kinematics.InverseKinematics_Ok AND RobotState = 130 THEN
399             ATS0(angle_deg := 90.0 - th1, steps_per_rev := 6400.0,
    gear_ratio := 50.0);
400             ATS1(angle_deg := 90.0 - th2, steps_per_rev := 6400.0,
    gear_ratio := 50.0);
401             ATS2(angle_deg := 90.0 - th3, steps_per_rev := 6400.0,
    gear_ratio := 50.0);
402
403             Motor0_Steps := ATS0.AngleToSteps - Prev_Motor0_Steps;
404             Motor1_Steps := ATS1.AngleToSteps - Prev_Motor1_Steps;
405             Motor2_Steps := ATS2.AngleToSteps - Prev_Motor2_Steps;
406
```

```
407             Prev_Motor0_Steps := ATS0.AngleToSteps;
408             Prev_Motor1_Steps := ATS1.AngleToSteps;
409             Prev_Motor2_Steps := ATS2.AngleToSteps;
410 
411             RobotState := 140;  // Move to Safe DropOff
412             ELSE
413             RobotState := 300;
414         END_IF;
415 
416         //------- 140. Move to Safe DropOff --------------------------------
417         140: // Trigger Move (Ramp Start)
418         StartPos0 := EIP_X_Motion.Motor0_CommandedPosition;
419         StartPos1 := EIP_X_Motion.Motor1_CommandedPosition;
420         StartPos2 := EIP_X_Motion.Motor2_CommandedPosition;
421 
422         EIP_Y_Motion.Motor0_MoveDistance := Motor0_Steps;
423         EIP_Y_Motion.Motor1_MoveDistance := Motor1_Steps;
424         EIP_Y_Motion.Motor2_MoveDistance := Motor2_Steps;
425         EIP_Y_Motion.Motor0_LoadPositionMove := TRUE;
426         EIP_Y_Motion.Motor1_LoadPositionMove := TRUE;
427         EIP_Y_Motion.Motor2_LoadPositionMove := TRUE;
428         RobotState := 141;
429 
430         141: // Wait Ack
431         IF EIP_X_Motion.Motor0_LoadPositionMoveAck AND
    EIP_X_Motion.Motor1_LoadPositionMoveAck AND
    EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
432             EIP_Y_Motion.Motor0_LoadPositionMove := FALSE;
433             EIP_Y_Motion.Motor1_LoadPositionMove := FALSE;
434             EIP_Y_Motion.Motor2_LoadPositionMove := FALSE;
435             RobotState := 142;
436         END_IF;
437 
438         142: // Wait Ack Clear
439         IF NOT EIP_X_Motion.Motor0_LoadPositionMoveAck AND NOT
    EIP_X_Motion.Motor1_LoadPositionMoveAck AND NOT
    EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
440             RobotState := 143;
441         END_IF;
442 
443         143: // Ramp Monitor (Blend to Drop)
444         DistTotal := ABS(Motor0_Steps) + ABS(Motor1_Steps) +
    ABS(Motor2_Steps);
445         DistDone  := ABS(EIP_X_Motion.Motor0_CommandedPosition - StartPos0) +
446         ABS(EIP_X_Motion.Motor1_CommandedPosition - StartPos1) +
447         ABS(EIP_X_Motion.Motor2_CommandedPosition - StartPos2);
448 
```

```
449         IF (DistTotal > 0 AND (DistDone * 100 / DistTotal) >= RampPercent)
    OR DistTotal = 0 THEN
450             RobotState := 150;  // Compute IK for Actual DropOff
451         END_IF;
452
453         //------- 150. Compute IK for Actual DropOff
    --------------------------------
454         150:
455         IF ActiveFrame.PartClass[loopIndex] = 1 THEN    // Scrap
456             Inverse_Kinematics(
457             MyRobot := g_delta_robot,
458             x_0     := 50.0,
459             y_0     := -120.0,
460             z_0     := Z_Pick,
461             theta1  := th1, theta2  := th2, theta3  := th3
462             );
463             ELSIF ActiveFrame.PartClass[loopIndex] = 2 THEN // Rework
464             Inverse_Kinematics(
465             MyRobot := g_delta_robot,
466             x_0     := -50.0,
467             y_0     := -120.0,
468             z_0     := Z_Pick,
469             theta1  := th1, theta2  := th2, theta3  := th3
470             );
471             ELSE
472             RobotState := 300;
473         END_IF;
474
475         IF Inverse_Kinematics.InverseKinematics_Ok AND RobotState = 150 THEN
476             ATS0(angle_deg := 90.0 - th1, steps_per_rev := 6400.0,
    gear_ratio := 50.0);
477             ATS1(angle_deg := 90.0 - th2, steps_per_rev := 6400.0,
    gear_ratio := 50.0);
478             ATS2(angle_deg := 90.0 - th3, steps_per_rev := 6400.0,
    gear_ratio := 50.0);
479
480             Motor0_Steps := ATS0.AngleToSteps - Prev_Motor0_Steps;
481             Motor1_Steps := ATS1.AngleToSteps - Prev_Motor1_Steps;
482             Motor2_Steps := ATS2.AngleToSteps - Prev_Motor2_Steps;
483
484             Prev_Motor0_Steps := ATS0.AngleToSteps;
485             Prev_Motor1_Steps := ATS1.AngleToSteps;
486             Prev_Motor2_Steps := ATS2.AngleToSteps;
487
488             RobotState := 160;  // Move to Actual DropOff
489             ELSE
490             RobotState := 300;
```

```
491         END_IF;
492
493         //------- 160. Move to Actual DropOff
    --------------------------------
494         160: // Trigger Move
495         EIP_Y_Motion.Motor0_MoveDistance := Motor0_Steps;
496         EIP_Y_Motion.Motor1_MoveDistance := Motor1_Steps;
497         EIP_Y_Motion.Motor2_MoveDistance := Motor2_Steps;
498         EIP_Y_Motion.Motor0_LoadPositionMove := TRUE;
499         EIP_Y_Motion.Motor1_LoadPositionMove := TRUE;
500         EIP_Y_Motion.Motor2_LoadPositionMove := TRUE;
501         RobotState := 161;
502
503         161: // Wait Ack
504         IF EIP_X_Motion.Motor0_LoadPositionMoveAck AND
    EIP_X_Motion.Motor1_LoadPositionMoveAck AND
    EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
505             EIP_Y_Motion.Motor0_LoadPositionMove := FALSE;
506             EIP_Y_Motion.Motor1_LoadPositionMove := FALSE;
507             EIP_Y_Motion.Motor2_LoadPositionMove := FALSE;
508             RobotState := 162;
509         END_IF;
510
511         162: // Wait Ack Clear
512         IF NOT EIP_X_Motion.Motor0_LoadPositionMoveAck AND NOT
    EIP_X_Motion.Motor1_LoadPositionMoveAck AND NOT
    EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
513             RobotState := 163;
514         END_IF;
515
516         163: // Wait Completion (Full Stop for Drop)
517         IF NOT EIP_X_Motion.Motor0_StepsActive AND
    EIP_X_Motion.Motor0_AtTargetPosition AND
    EIP_X_Motion.Motor1_AtTargetPosition AND
    EIP_X_Motion.Motor2_AtTargetPosition THEN
518             RobotState := 170; // Drop Part
519         END_IF;
520
521         //------- 170. Drop Part ---------------------------------------
522         170:
523         PlaceComplete := TRUE;
524         TurnOnElectromagnet_Auto := FALSE;
525         IF ActiveFrame.PartClass[loopIndex] = 1 THEN
526             CurrentTCP.X_Position := 50.0;
527             ELSE
528             CurrentTCP.X_Position := -50.0;
529         END_IF;
```

```
530         CurrentTCP.Y_Position := -120.0;
531         CurrentTCP.Z_Position := Z_Pick;
532         RobotState := 180;  // Compute IK for Safe DropOff Lift
533
534         //------- 180. Compute IK for Safe DropOff Lift
    --------------------------------
535         180:
536         IF ActiveFrame.PartClass[loopIndex] = 1 THEN    // Scrap
537             Inverse_Kinematics(
538             MyRobot := g_delta_robot,
539             x_0     := 50.0,
540             y_0     := -120.0,
541             z_0     := Z_Safe,
542             theta1  := th1, theta2  := th2, theta3  := th3
543             );
544             ELSIF ActiveFrame.PartClass[loopIndex] = 2 THEN // Rework
545             Inverse_Kinematics(
546             MyRobot := g_delta_robot,
547             x_0     := -50.0,
548             y_0     := -120.0,
549             z_0     := Z_Safe,
550             theta1  := th1, theta2  := th2, theta3  := th3
551             );
552             ELSE
553             RobotState := 300;
554         END_IF;
555
556         IF Inverse_Kinematics.InverseKinematics_Ok AND RobotState = 180 THEN
557             ATS0(angle_deg := 90.0 - th1, steps_per_rev := 6400.0,
    gear_ratio := 50.0);
558             ATS1(angle_deg := 90.0 - th2, steps_per_rev := 6400.0,
    gear_ratio := 50.0);
559             ATS2(angle_deg := 90.0 - th3, steps_per_rev := 6400.0,
    gear_ratio := 50.0);
560
561             Motor0_Steps := ATS0.AngleToSteps - Prev_Motor0_Steps;
562             Motor1_Steps := ATS1.AngleToSteps - Prev_Motor1_Steps;
563             Motor2_Steps := ATS2.AngleToSteps - Prev_Motor2_Steps;
564
565             Prev_Motor0_Steps := ATS0.AngleToSteps;
566             Prev_Motor1_Steps := ATS1.AngleToSteps;
567             Prev_Motor2_Steps := ATS2.AngleToSteps;
568
569             RobotState := 190;  // Move to Safe DropOff Lift
570             ELSE
571             RobotState := 300;
572         END_IF;
```

```
573
574         //------- 190. Move to Safe DropOff Lift
    --------------------------------
575         190: // Trigger Move (Ramp Start)
576         StartPos0 := EIP_X_Motion.Motor0_CommandedPosition;
577         StartPos1 := EIP_X_Motion.Motor1_CommandedPosition;
578         StartPos2 := EIP_X_Motion.Motor2_CommandedPosition;
579
580         EIP_Y_Motion.Motor0_MoveDistance := Motor0_Steps;
581         EIP_Y_Motion.Motor1_MoveDistance := Motor1_Steps;
582         EIP_Y_Motion.Motor2_MoveDistance := Motor2_Steps;
583         EIP_Y_Motion.Motor0_LoadPositionMove := TRUE;
584         EIP_Y_Motion.Motor1_LoadPositionMove := TRUE;
585         EIP_Y_Motion.Motor2_LoadPositionMove := TRUE;
586         RobotState := 191;
587
588         191: // Wait Ack
589         IF EIP_X_Motion.Motor0_LoadPositionMoveAck AND
    EIP_X_Motion.Motor1_LoadPositionMoveAck AND
    EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
590             EIP_Y_Motion.Motor0_LoadPositionMove := FALSE;
591             EIP_Y_Motion.Motor1_LoadPositionMove := FALSE;
592             EIP_Y_Motion.Motor2_LoadPositionMove := FALSE;
593             RobotState := 192;
594         END_IF;
595
596         192: // Wait Ack Clear
597         IF NOT EIP_X_Motion.Motor0_LoadPositionMoveAck AND NOT
    EIP_X_Motion.Motor1_LoadPositionMoveAck AND NOT
    EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
598             RobotState := 193;
599         END_IF;
600
601         193: // Ramp Monitor (Blend to Next)
602         DistTotal := ABS(Motor0_Steps) + ABS(Motor1_Steps) +
    ABS(Motor2_Steps);
603         DistDone  := ABS(EIP_X_Motion.Motor0_CommandedPosition - StartPos0) +
604         ABS(EIP_X_Motion.Motor1_CommandedPosition - StartPos1) +
605         ABS(EIP_X_Motion.Motor2_CommandedPosition - StartPos2);
606
607         IF (DistTotal > 0 AND (DistDone * 100 / DistTotal) >= RampPercent)
    OR DistTotal = 0 THEN
608             RobotState := 200;
609         END_IF;
610
611         //------- 200. Return Home / Next Part
    ---------------------------------------
```

```
612         200:
613         loopIndex := loopIndex + 1;
614         IF loopIndex >= PartsTotal THEN
615             // Frame Complete
616             RobotState := 210;  // Absolute Move Home
617             ELSE
618             RobotState := 220;
619         END_IF;
620
621         //------- 210. Home Correction (Absolute Move) ----------------------------------------
622         210:
623         // Wait for motors to be fully stopped before correction to prevent jumps
624         IF NOT EIP_X_Motion.Motor0_StepsActive AND EIP_X_Motion.Motor0_AtTargetPosition AND
625             NOT EIP_X_Motion.Motor1_StepsActive AND EIP_X_Motion.Motor1_AtTargetPosition AND
626             NOT EIP_X_Motion.Motor2_StepsActive AND EIP_X_Motion.Motor2_AtTargetPosition THEN
627
628             // Return to Home (26667 steps)
629             EIP_Y_Motion.Motor0_MoveDistance := 26667 - Prev_Motor0_Steps;
630             EIP_Y_Motion.Motor1_MoveDistance := 26667 - Prev_Motor1_Steps;
631             EIP_Y_Motion.Motor2_MoveDistance := 26667 - Prev_Motor2_Steps;
632
633             Prev_Motor0_Steps := 26667;
634             Prev_Motor1_Steps := 26667;
635             Prev_Motor2_Steps := 26667;
636
637             EIP_Y_Motion.Motor0_LoadPositionMove := TRUE;
638             EIP_Y_Motion.Motor1_LoadPositionMove := TRUE;
639             EIP_Y_Motion.Motor2_LoadPositionMove := TRUE;
640
641             MotionStarted := FALSE;
642             AllAtTarget := FALSE;
643             RobotState := 211;
644         END_IF;
645
646         211:
647         IF EIP_X_Motion.Motor0_LoadPositionMoveAck AND EIP_X_Motion.Motor1_LoadPositionMoveAck AND
    EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
648             EIP_Y_Motion.Motor0_LoadPositionMove := FALSE;
649             EIP_Y_Motion.Motor1_LoadPositionMove := FALSE;
650             EIP_Y_Motion.Motor2_LoadPositionMove := FALSE;
651             RobotState := 212;
```

```
652         END_IF;
653
654         212:
655         IF NOT EIP_X_Motion.Motor0_LoadPositionMoveAck AND NOT
    EIP_X_Motion.Motor1_LoadPositionMoveAck AND NOT
    EIP_X_Motion.Motor2_LoadPositionMoveAck THEN
656             MotionStarted := TRUE;
657             RobotState := 213;
658         END_IF;
659
660         213:
661         IF NOT EIP_X_Motion.Motor0_StepsActive AND
    EIP_X_Motion.Motor0_AtTargetPosition AND
    EIP_X_Motion.Motor1_AtTargetPosition AND
    EIP_X_Motion.Motor2_AtTargetPosition THEN
662             RobotState := 220;  // Check Completion
663         END_IF;
664
665         //------- 220. Check Completion
    ----------------------------------------
666         220:
667         IF loopIndex >= PartsTotal THEN
668             RobotState := 10;   // Main Loop (Idle)
669             ELSE
670             RobotState := 30;   // Step-Thru Main Loop
671         END_IF;
672
673     END_CASE;
674
675     ELSE
676     AutoMode := FALSE;
677     ConveyorRun_Auto := FALSE;
678     TurnOnElectromagnet_Auto := FALSE;
679     RobotState := 0;
680     VisionFrameID := 0;
681     WriteIdx := 0;
682     ReadIdx := 0;
683     FrameCount := 0;
684 END_IF;
685
```

Function Block

```
base_y := -MyRobot.Base_Radius;

CA1(
Robot  := MyRobot,
x_0    := x_0,
y_0    := y_0,
z_0    := z_0,
base_y := base_y
);

IF NOT CA1.Ok THEN
    InverseKinematics_Ok := FALSE;
    RETURN;
END_IF;

theta1 := CA1.theta;

x_1 := x_0 * MyRobot.cos120 + y_0 * MyRobot.sin120;
y_1 := y_0 * MyRobot.cos120 - x_0 * MyRobot.sin120;
base_y := -MyRobot.Base_Radius; // * MyRobot.cos120;

CA2(
Robot  := MyRobot,
x_0    := x_1,
y_0    := y_1,
z_0    := z_0,
base_y := base_y
);

IF NOT CA2.Ok THEN
    InverseKinematics_Ok := FALSE;
    RETURN;
END_IF;

theta2 := CA2.theta;

x_2 := x_0 * MyRobot.cos120 - y_0 * MyRobot.sin120;
y_2 := y_0 * MyRobot.cos120 + x_0 * MyRobot.sin120;
base_y := -MyRobot.Base_Radius; // * MyRobot.cos120;

CA3(
Robot  := MyRobot,
x_0    := x_2,
y_0    := y_2,
z_0    := z_0,
base_y := base_y
```

Function Block

```
47 );
48
49 IF NOT CA3.Ok THEN
50     InverseKinematics_Ok := FALSE;
51     RETURN;
52 END_IF;
53
54 theta3 := CA3.theta;
55
56 theta1 := theta1 * 180.0 / PI;
57 theta2 := theta2 * 180.0 / PI;
58 theta3 := theta3 * 180.0 / PI;
59
60 InverseKinematics_Ok := TRUE;
61
```

Function Block

```
1 AngleToSteps := REAL_TO_DINT((angle_deg / 360.0) * steps_per_rev *
  gear_ratio);
```

Function Block

```
IF z_0 = 0.0 THEN
    Ok := FALSE;
    RETURN;
END_IF;

y_1 := base_y;
y_0_local := y_0 - Robot.t_1;

a_1 :=
(x_0*x_0 + y_0_local*y_0_local + z_0*z_0
+ Robot.Bicep_Length*Robot.Bicep_Length
- Robot.Forearm_Length*Robot.Forearm_Length
- y_1*y_1)
/ (2.0 * z_0);

b_1 := (y_1 - y_0_local) / z_0;

d_1 :=
-(a_1 + b_1*y_1)*(a_1 + b_1*y_1)
+ Robot.Bicep_Length*Robot.Bicep_Length*(b_1*b_1 + 1.0);

IF d_1 < 0.0 THEN
    Ok := FALSE;
    RETURN;
END_IF;

yj := (y_1 - a_1*b_1 - SQRT(d_1)) / (b_1*b_1 + 1.0);
zj := a_1 + b_1*yj;

IF (y_1 - yj) = 0.0 THEN
    Ok := FALSE;
    RETURN;
END_IF;

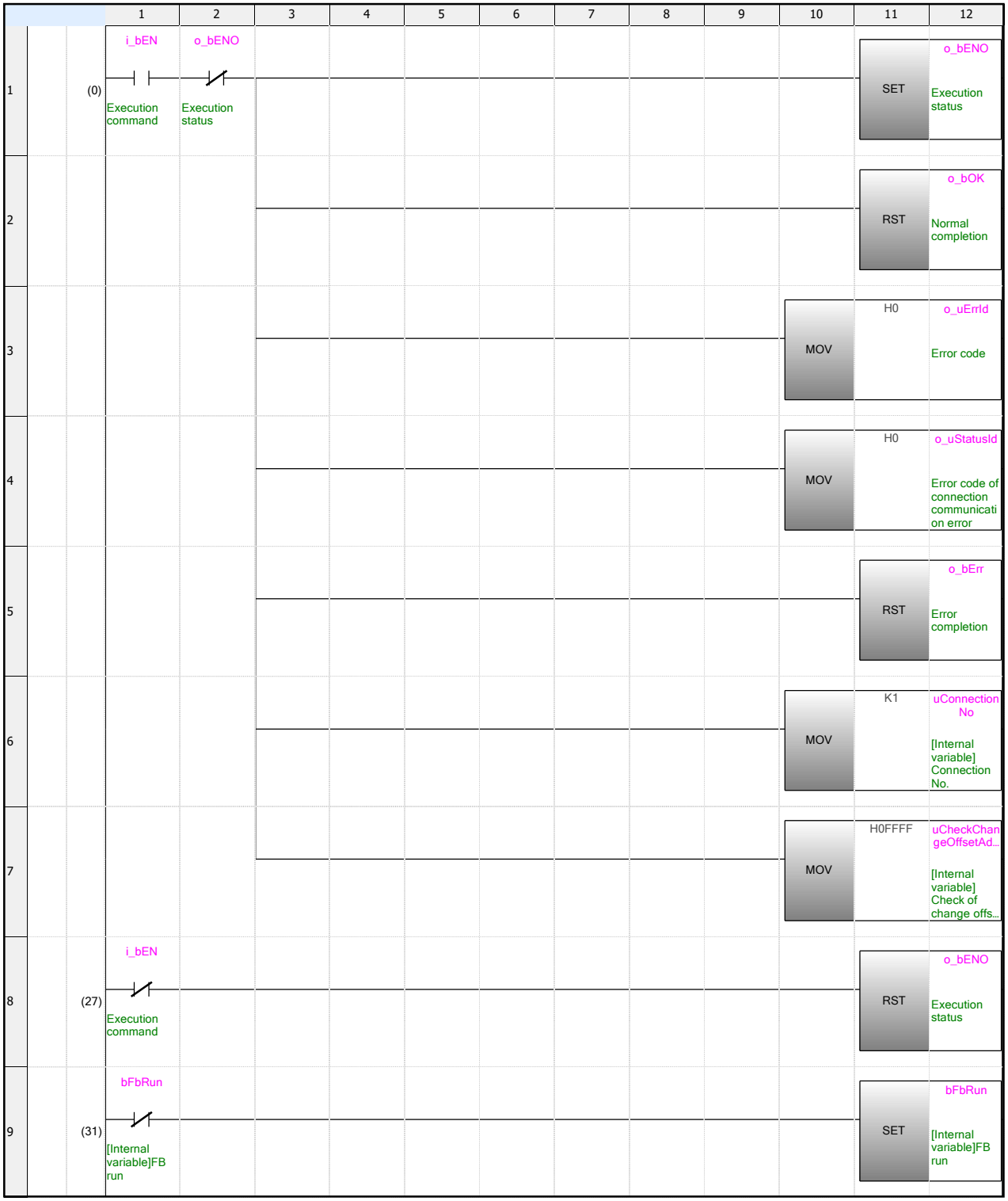
theta := ATAN(-zj / (y_1 - yj));

//IF yj > y_1 THEN
//    theta := theta + PI;
//END_IF;

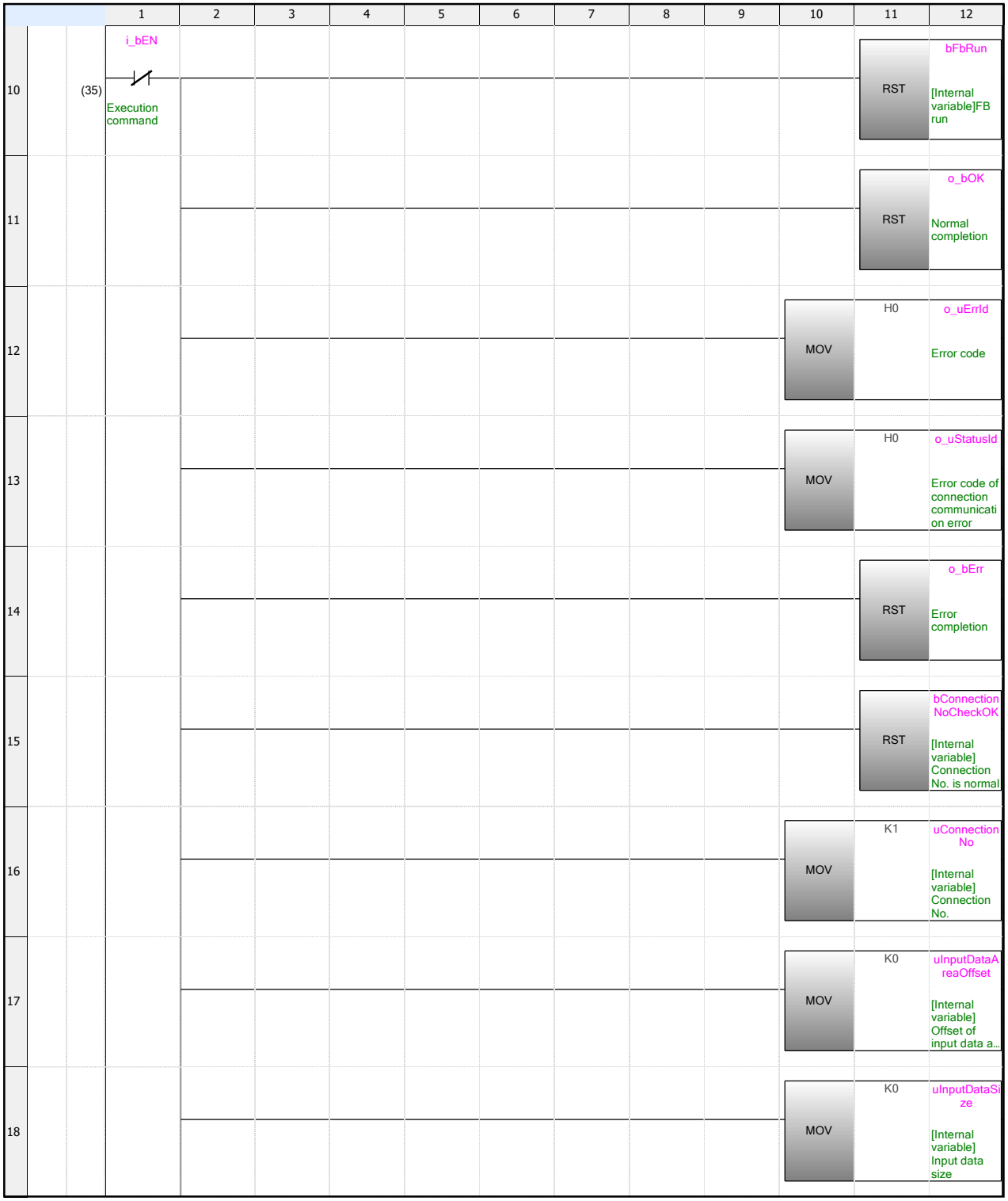
Ok := TRUE;

```

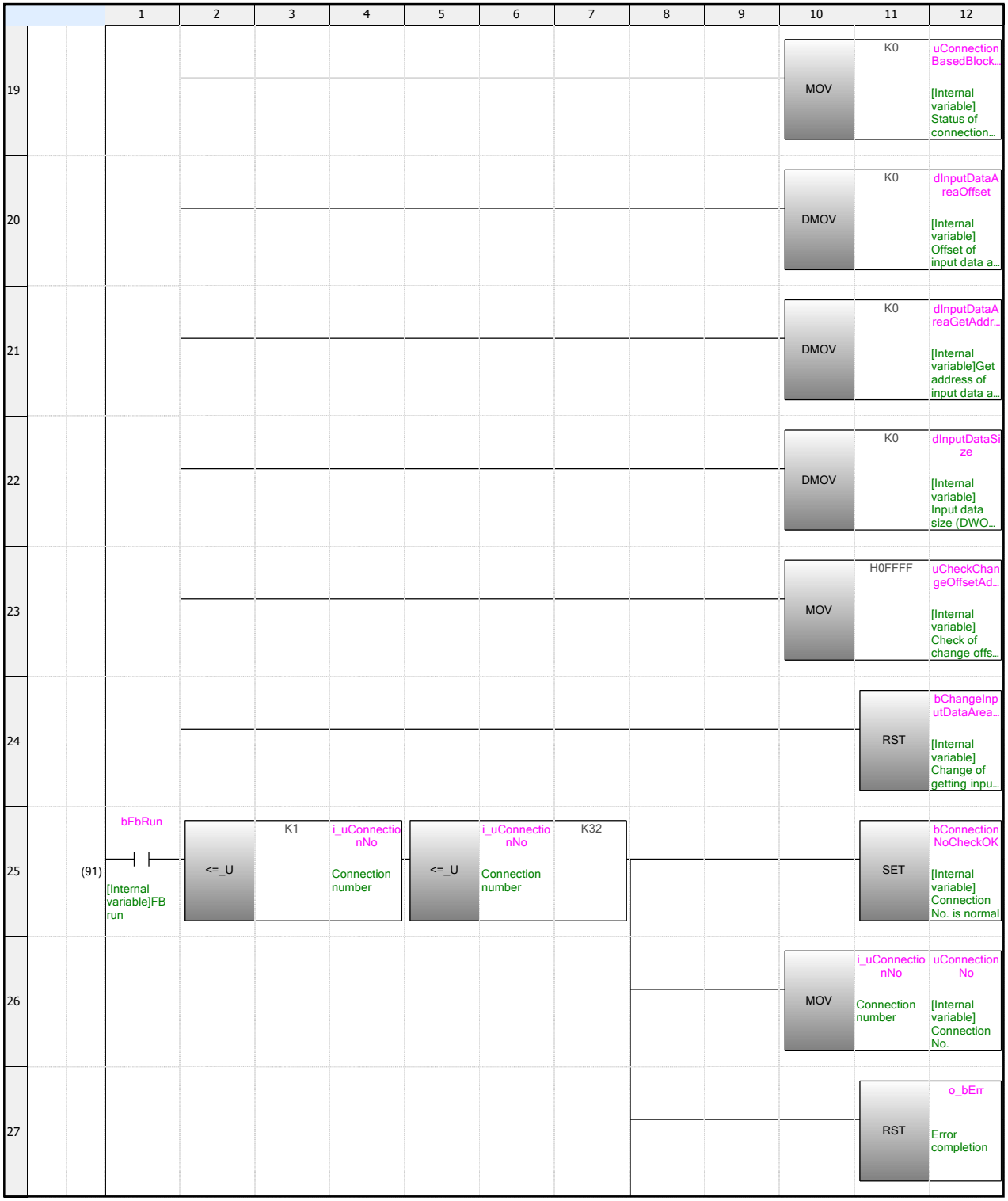
Function Block



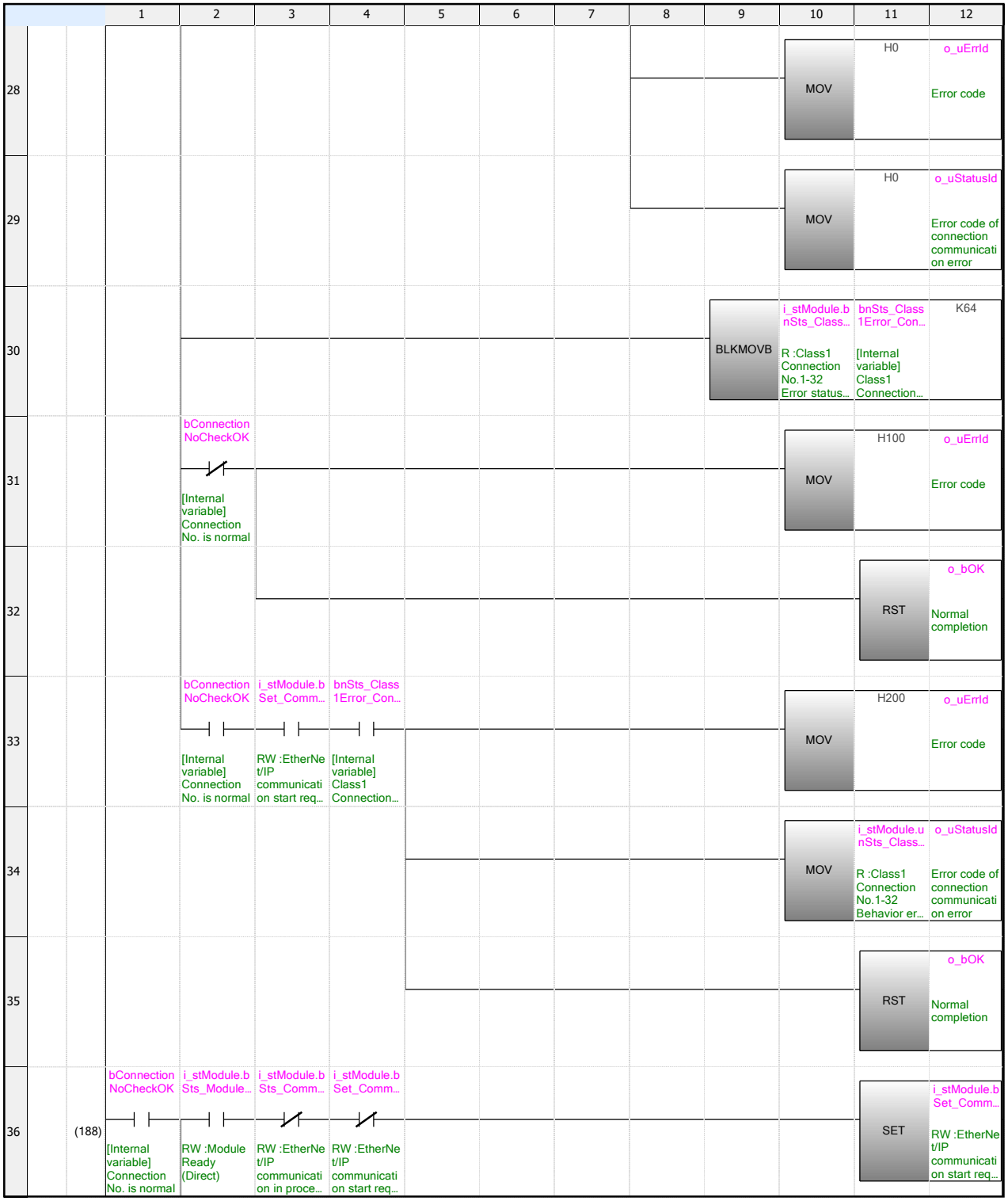
Function Block



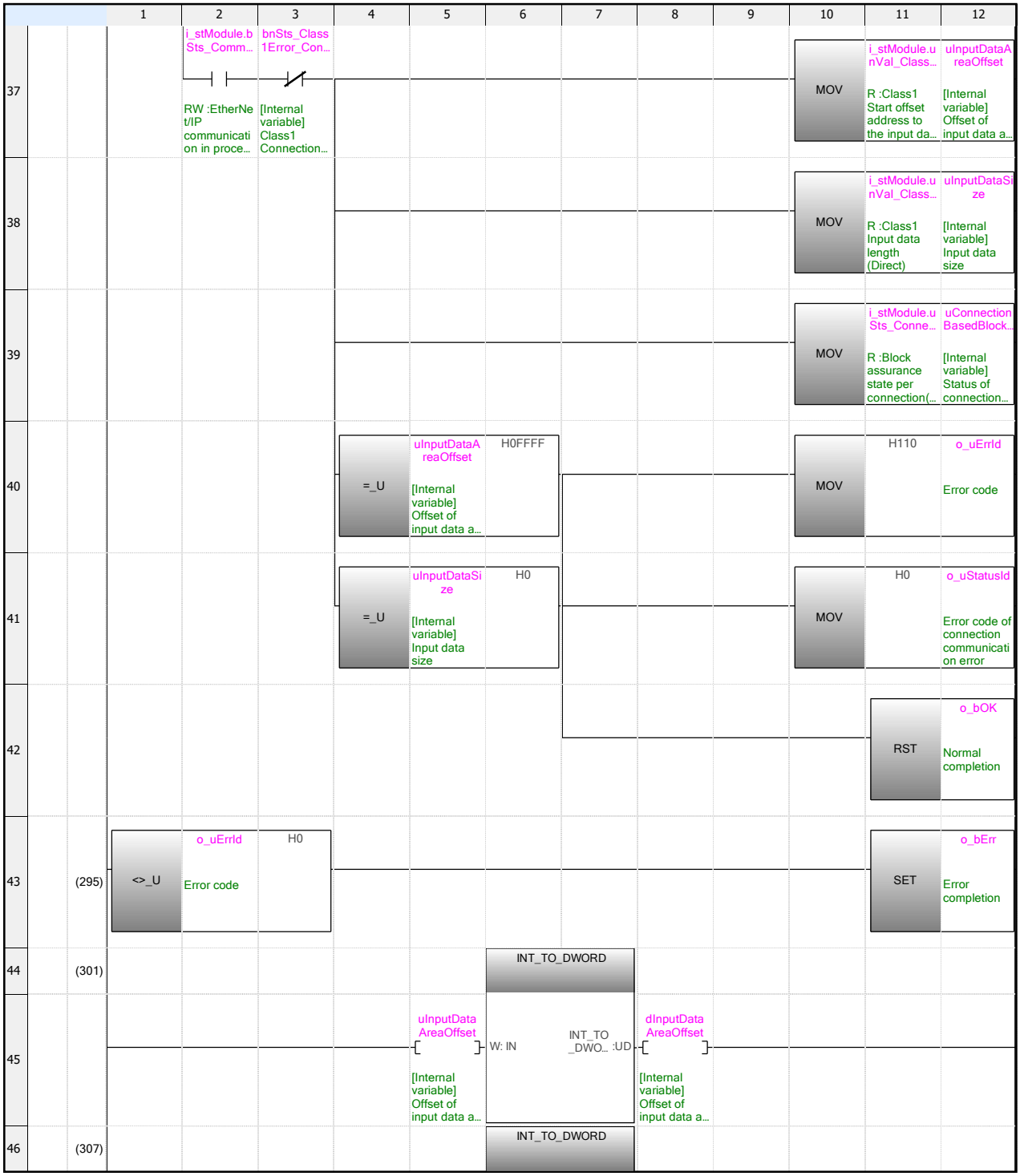
Function Block



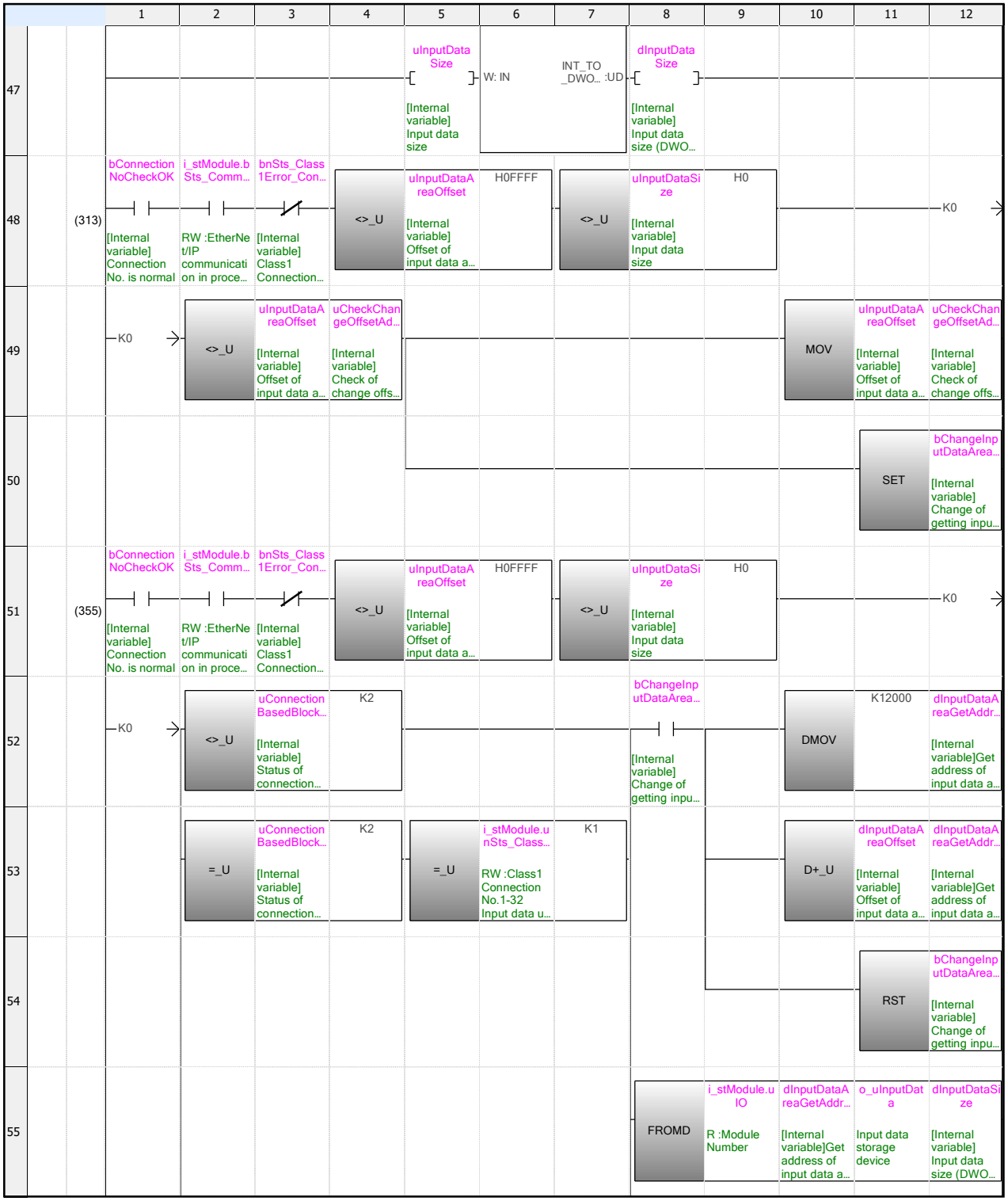
Function Block



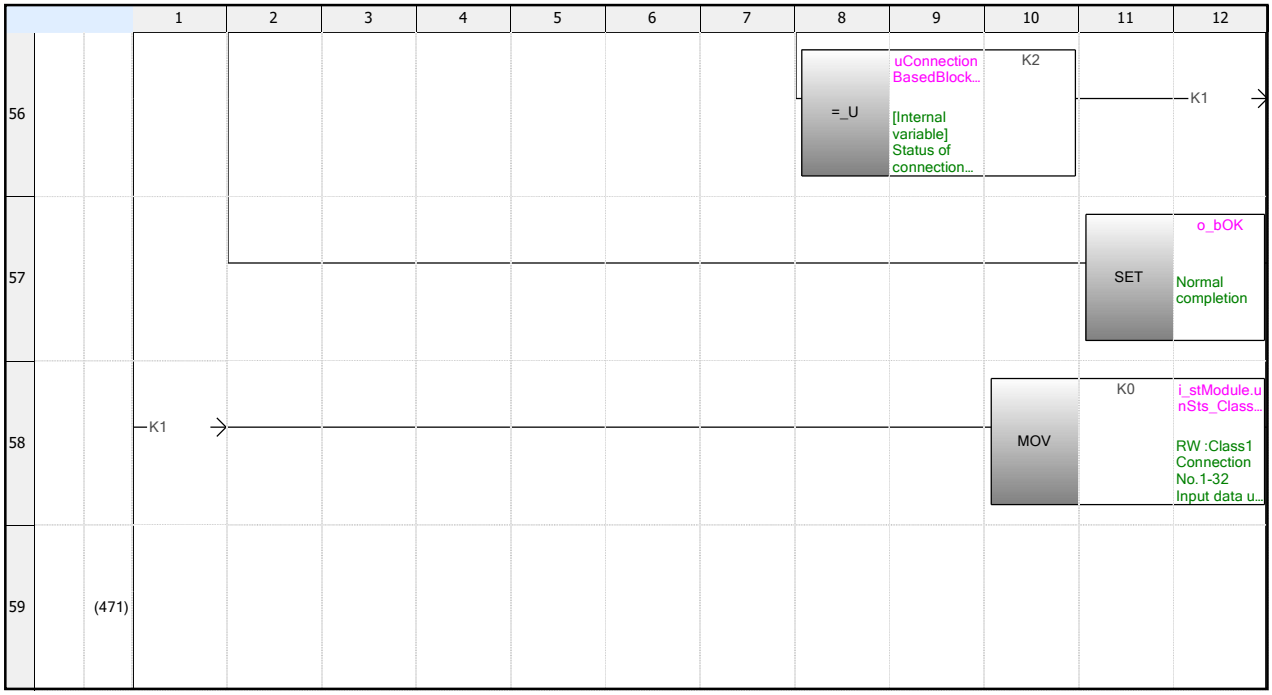
Function Block



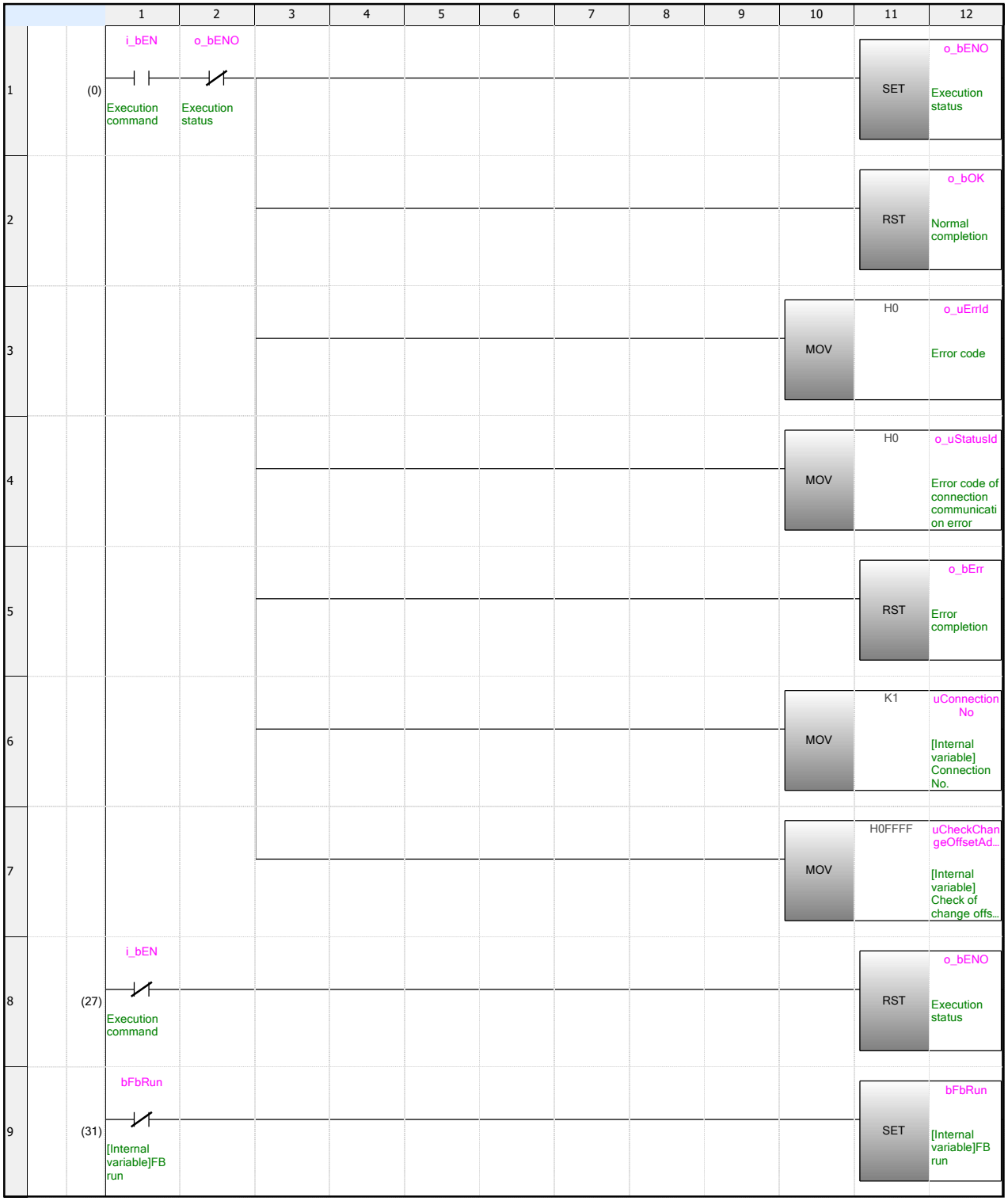
Function Block



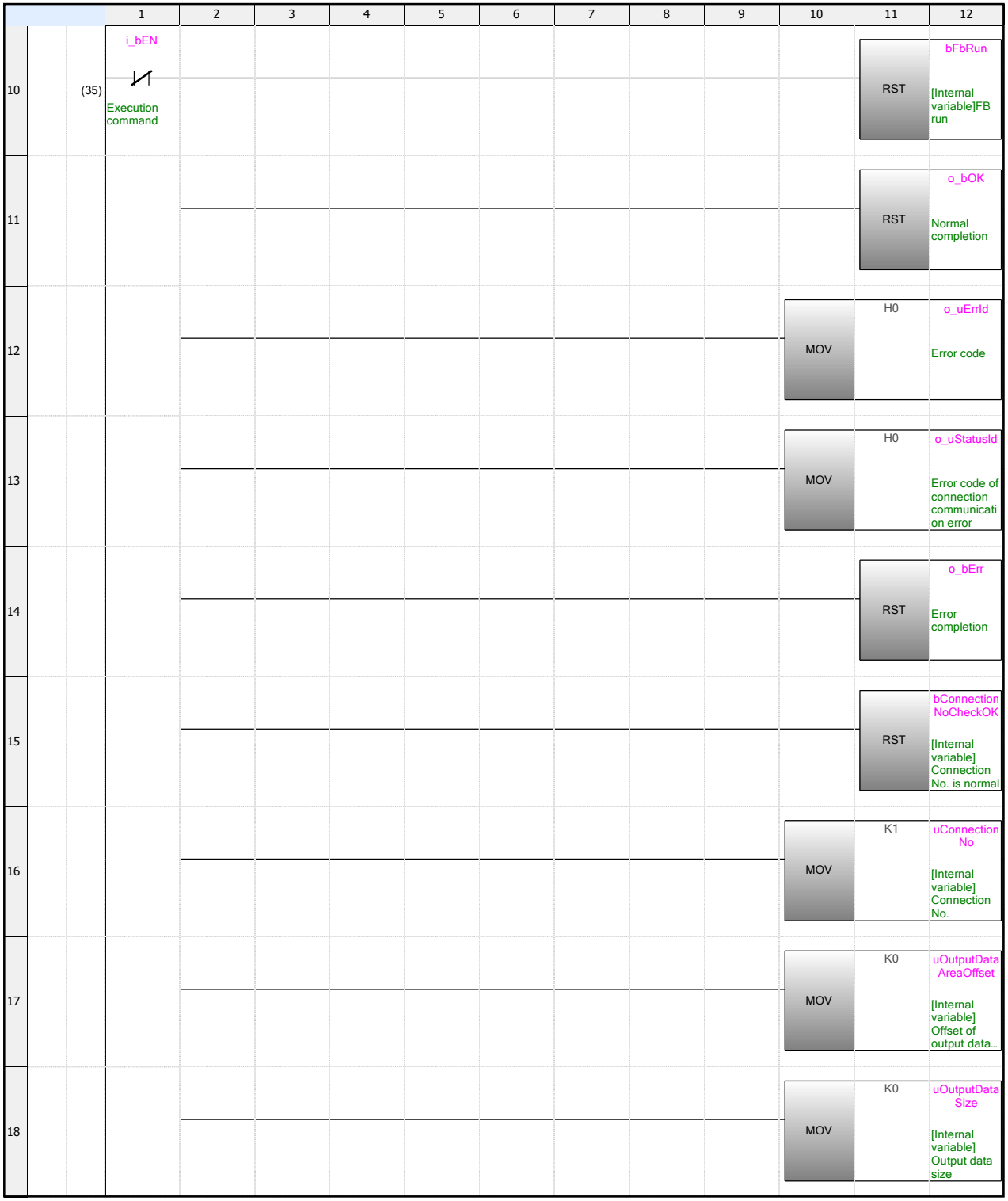
Function Block



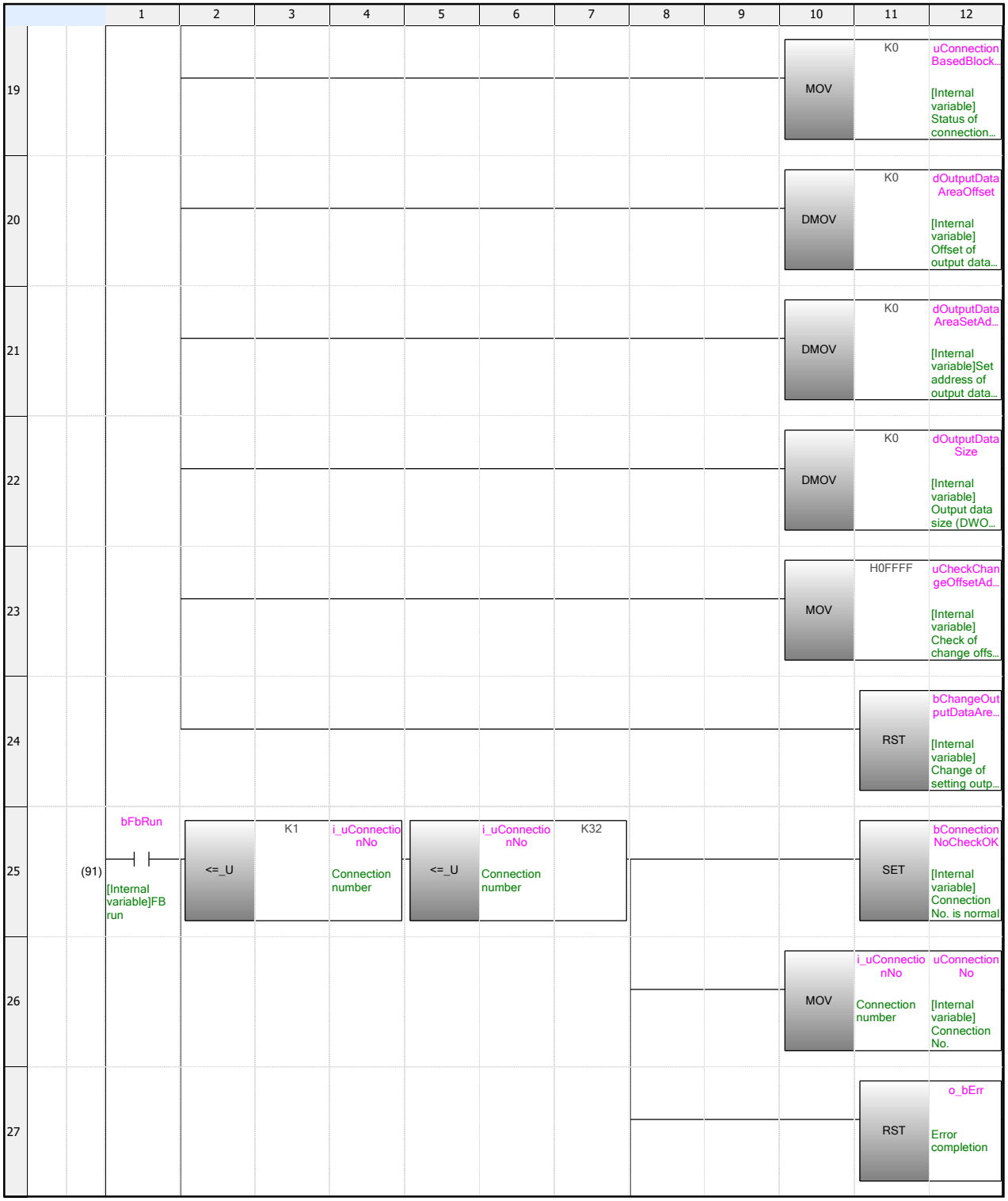
Function Block



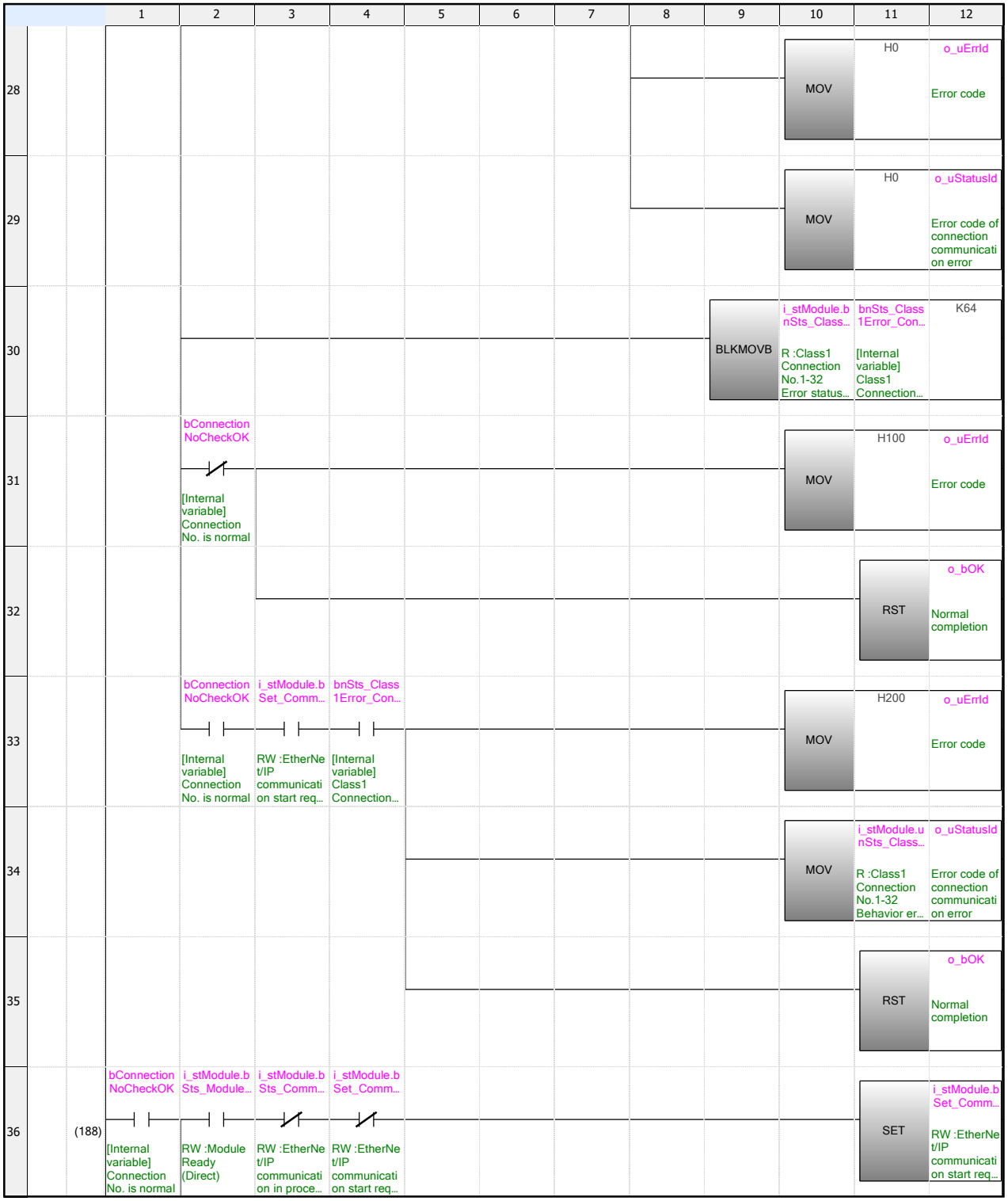
Function Block



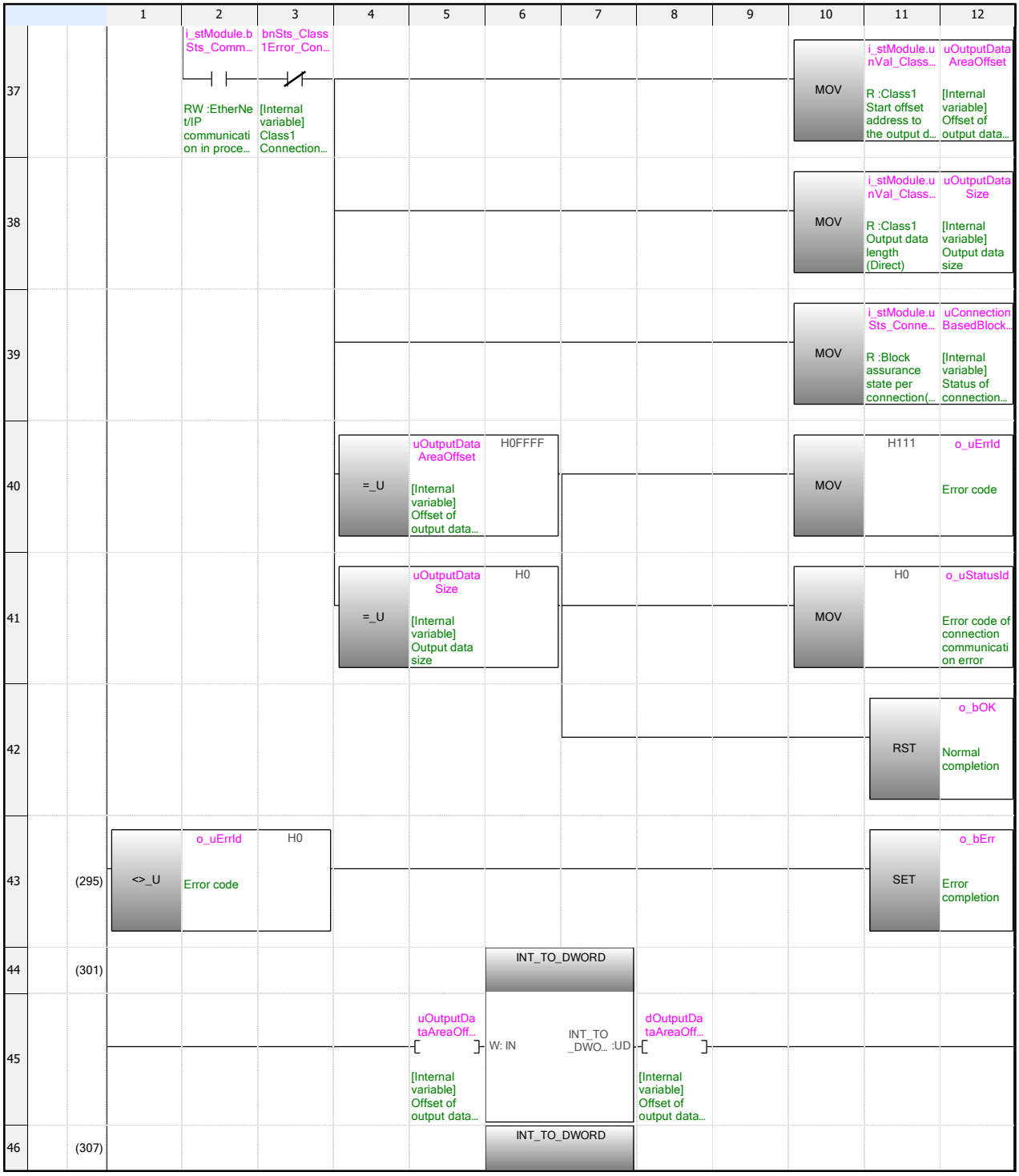
Function Block



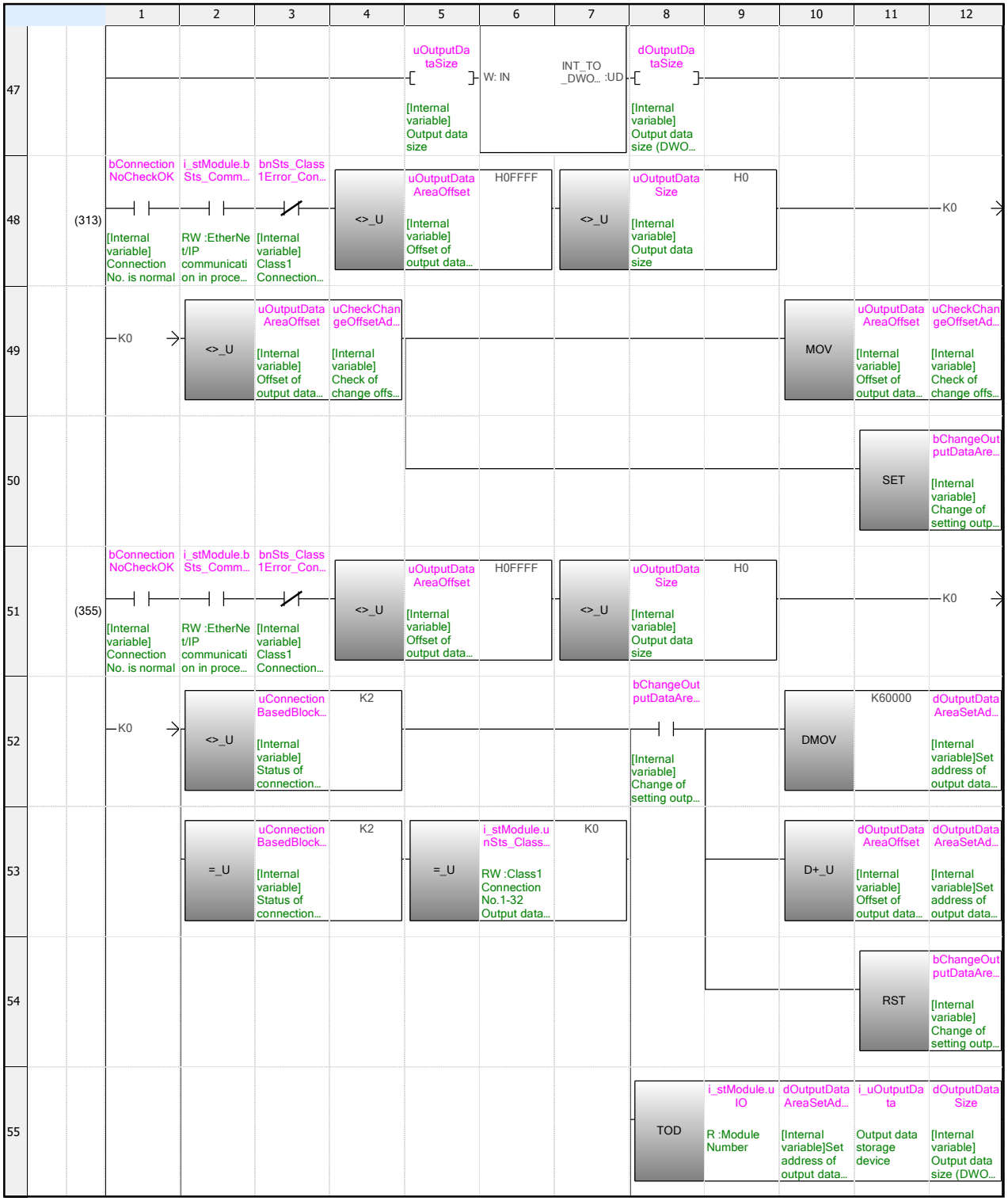
Function Block



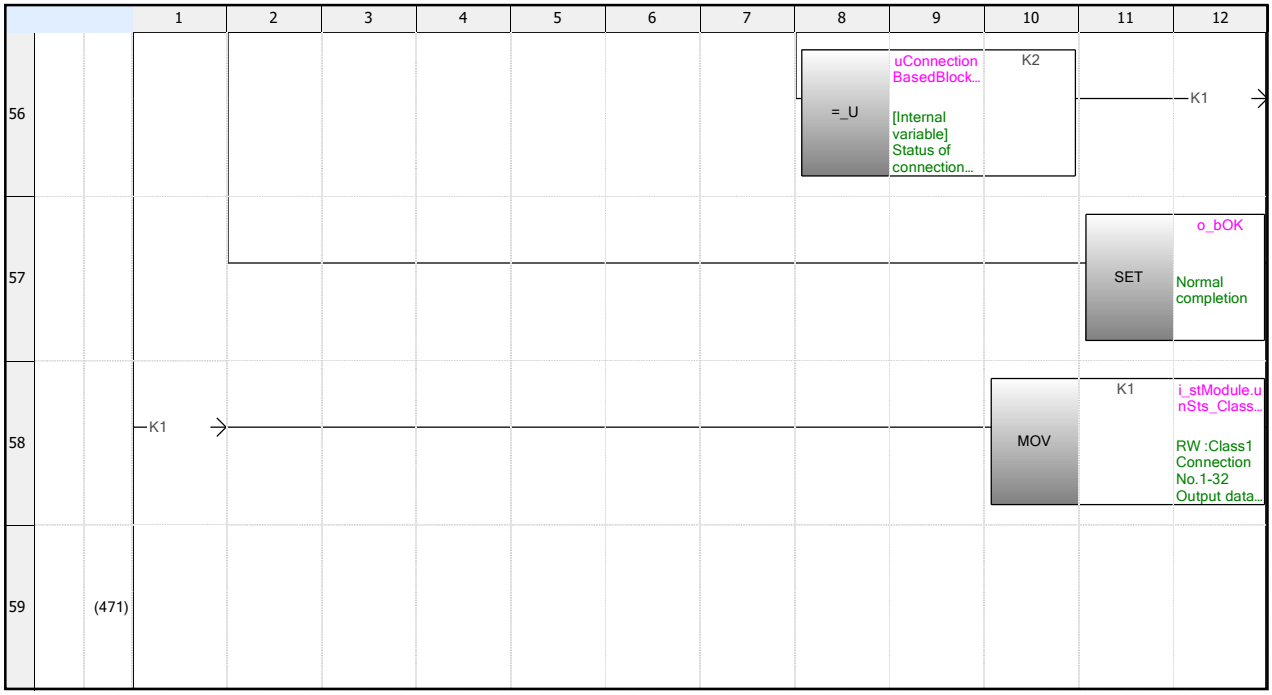
Function Block



Function Block



Function Block



Basic Settings

Item	Setting
Own Node Settings	
IP Address	
IP Address	10 . 10 . 82 . 1
Subnet Mask	255 . 255 . 255 . 0
Default Gateway	. . .
Communication Data Code	Binary
CC-Link IEF Basic Settings	
To Use or Not to Use CC-Link IEF Basic Setting	Not to Use
Network Configuration Settings	<Detailed Setting>
Refresh Settings	<Detailed Setting>
MODBUS/TCP Settings	
To Use or Not to Use MODBUS/TCP Setting	Not Used
Device Assignment	<Detailed Setting>
External Device Configuration	
External Device Configuration	<Detailed Setting>
Transmission Port Settings	
MELSOFT Transmission Port UDP/IP	Use
MELSOFT Transmission Port TCP/IP	Use

Basic Settings

External Device Configuration

No.	Model Name	Communication Measure	Protocol	Fixed Buffer Send/Receive Setting	PLC			Sensor/Device					Existence Confirmation
					IP Address	Port No.	MAC Address	Host Name	IP Address	Port No.	Subnet Mask	Default Gateway	
	Host Station				10.10.82.1								
1	MELSOFT Connection Module	MELSOFT Connection	TCP		10.10.82.1								KeepAlive
2	MELSOFT Connection Module	MELSOFT Connection	TCP		10.10.82.1								KeepAlive

Application Settings

Item	Setting
FTP Server Settings	
FTP Server	Not to Use
Login Name	FXCPU
Advanced Settings	
Password Setting	
Current Password	
New Password	
Confirm New Password	
FTP Timer Settings	
Command Input Monitoring Timer	900
Unit	s
Response Monitoring Timer	5
Unit	s
Allow Online Change	Disable
FTP Client Settings	
To Use or Not to Use FTP Client Settings	Not to Use
FTP Server Specification	FTP Server Address
FTP Server IP Address	0 . 0 . 0 . 1
Login Name	
Password	
Connection Method	PORT Mode
Port No.	21
Web Server Settings	
To Use or Not to Use Web Server Settings	Not to Use
Host Station Port No.	80
Account Settings	<Detailed Setting>
Simple CPU Communication Setting	
To Use or Not to Use Simple CPU Communication	Not to Use
Simple CPU Communication Setting	<Detailed Setting>
Time Setting	
Time Setting (SNTP Client)	Not Use
SNTP Server IP Address	0 . 0 . 0 . 1
Timer Setting After Power-on and Reset	Disable
Execution Timing	Fixed Time
Time Intervals	1 Minute
Specified Time (Hour, Minute, Day of Week)	
Clock Time (Hour, Minute)	
Hour	12
Minute	0
Day of Week	
Sun	Set
Mon	Set

Application Settings

Item	Setting
Tue	Set
Wed	Set
Thu	Set
Fri	Set
Sat	Set
Security	
IP Filter Settings	
IP Filter	Disable
IP Filter Settings	<Detailed Setting>
Disable Direct Connection with MELSOFT	
Disable Direct Connection with MELSOFT	Enable
Do Not Respond to CPU Module Search	
Do Not Respond to CPU Module Search	Respond
iQ Sensor Solution Settings	
Use iQ Sensor Solution	Use
SLMPSND Instruction Settings	
Use SLMPSND Instruction	Use

Module Parameter : Required Settings

Item	Setting
Mode	
Communication Mode	Online

Module Parameter : Basic Settings

Item	Setting
Own Node Settings	
IP Address	
IP Address	10 . 10 . 82 . 2
Subnet Mask	255 . 255 . 255 . 0
Default Gateway	10 . 10 . 82 . 1
Communication Data Code	Binary
Opening Method	Do Not Open by Program
Operational Setting	
Timer Settings for Data Communication	
Change/Set Timer Value	No
Destination Alive Check Start Interval Timer	600
Unit	s
Destination Alive Check Interval Timer	10
Unit	s
Destination Alive Check Resend Count	3 Times
BACnet Function Setting	
To Use or Not to Use BACnet Function	Not Used
Network Information Setting	
Network No.	0
BACnet Device Setting	
Instance No.	0
Object Name	
BACnet Standard	ANSI/ASHRAE Standard 135-2016
I-Am Send Setting	
Send Control	Send only When Device Status is Operational
Send Setting upon Device Status Change to Operational	Send
Enable/Disable Fixed Interval Send	Disable
Cycle of Fixed Interval Send [Second]	60 Second
I-Am Response Setting	LocalBroadcast
I-Am Send Setting	LocalBroadcast
Device Restart Procedures Setting	
Device Restart Procedures	Not to Support
Report Destination Network No.	0
Report Destination IP Address	192 . 168 . 0 . 254
Report Destination Port No.	47808
Time Synchronization Setting	Ignore
BACnet Object Setting	
Accumulator Object Setting	<Detailed Setting>
AnalogInput Object Setting	<Detailed Setting>
AnalogOutput Object Setting	<Detailed Setting>
AnalogValue Object Setting	<Detailed Setting>
BinaryInput Object Setting	<Detailed Setting>

Module Parameter : Basic Settings

Item	Setting
BinaryOutput Object Setting	<Detailed Setting>
BinaryValue Object Setting	<Detailed Setting>
Multi-state Input Object Setting	<Detailed Setting>
Multi-state Output Object Setting	<Detailed Setting>
External Device Configuration	
External Device Configuration	<Detailed Setting>

Module Parameter : Application Settings

Item	Setting
Simple CPU Communication Setting	
To Use or Not to Use Simple CPU Communication	Not to Use
CPU Response Monitoring Timer	5
Unit	s
Simple CPU Communication Setting	<Detailed Setting>
Security	
IP Filter Settings	
IP Filter	Disable
IP Filter Settings	<Detailed Setting>
Disable Direct Connection with MELSOFT	
Disable Direct Connection with MELSOFT	Enable
Do Not Respond to CPU Module Search	
Do Not Respond to CPU Module Search	Respond

Workspace Name :
Project Name :
Title :

Data Name	Last Change	Title
Project	4/30/2026 12:13:14 PM	
Module Configuration		
Program		
Initial		
Scan		
MAIN	4/30/2026 12:13:14 PM	
EIP_Read	4/30/2026 12:06:38 PM	
Local Label	4/10/2026 7:14:08 PM	
ProgramBody	4/30/2026 12:06:38 PM	
EIP_Write	4/30/2026 12:06:38 PM	
Local Label	4/10/2026 7:14:13 PM	
ProgramBody	4/30/2026 12:06:38 PM	
Vision	4/30/2026 12:06:38 PM	
Local Label	3/31/2026 10:21:04 AM	
ProgramBody	4/30/2026 12:06:38 PM	
Motion	4/30/2026 12:06:38 PM	
Local Label	3/19/2026 12:32:46 PM	
ProgramBody	4/30/2026 12:06:38 PM	
Conveyor	4/30/2026 12:06:38 PM	
Local Label	1/31/2026 11:42:08 PM	
ProgramBody	4/30/2026 12:06:38 PM	
Auto	4/30/2026 12:13:14 PM	
Local Label	4/14/2026 11:49:26 AM	
ProgramBody	4/30/2026 12:13:14 PM	
Fixed Scan		
Event		
Standby		
No Execution Type		
Unregistered Program		
FB/FUN	4/30/2026 12:06:37 PM	
InverseKinematics	4/30/2026 12:06:36 PM	
InverseKinematics	4/30/2026 12:06:36 PM	
Local Label	12/29/2025 12:28:24 AM	
ProgramBody	4/30/2026 12:06:36 PM	
AngleToSteps	4/30/2026 12:06:36 PM	
AngleToSteps	4/30/2026 12:06:36 PM	
Local Label	12/29/2025 9:05:00 PM	
ProgramBody	4/30/2026 12:06:36 PM	

Workspace Name :
Project Name :
Title :

Data Name	Last Change	Title
CalculateAngle_FB	4/30/2026 12:06:36 PM	
CalculateAngle_FB	4/30/2026 12:06:36 PM	
Local Label	12/29/2025 12:35:11 AM	
ProgramBody	4/30/2026 12:06:36 PM	
M_FBLIB	4/30/2026 12:06:37 PM	
M+FX5ENETIP_Class1GetInputData_01A	4/30/2026 12:06:37 PM	Get Class1 input data
Local Label	11/6/2025 1:01:11 PM	
ProgramBody	4/30/2026 12:06:37 PM	
M+FX5ENETIP_Class1SetOutputData_01A	4/30/2026 12:06:37 PM	Set Class1 Output data
Local Label	11/6/2025 1:07:51 PM	
ProgramBody	4/30/2026 12:06:37 PM	
Label		
Global Label	4/26/2026 7:26:10 PM	
VisionTags	4/26/2026 7:26:10 PM	
Main	4/13/2026 9:57:54 AM	
M+Global	11/6/2025 12:44:08 PM	
MotionTags	4/10/2026 8:26:11 PM	
ConveyorTags	2/19/2026 10:50:45 AM	
Structured Data Types	4/10/2026 7:15:43 PM	
M+FX5ENETIP	3/2/2021 7:17:41 PM	
M+FX5UCPU_CPU_Restoration_Function_Setting	7/5/2019 4:10:47 AM	
M+FX5UCPU_CPU_Restoration_Target_Date_Stamp_Folder_Setting	7/5/2019 4:10:56 AM	
M+FX5UCPU_Clock_Data	7/5/2019 4:10:22 AM	
M+FX5UCPU_Error_Information	7/5/2019 4:10:17 AM	
M+FX5UCPU_F	7/5/2019 4:10:12 AM	
M+FX5UCPU_Error_Time	7/5/2019 4:10:06 AM	
M+FX5UCPU_Latest_Error	6/4/2021 12:22:56 AM	
M+FX5UCPU_MemoryCard_Use_Condition	7/5/2019 4:10:39 AM	
M+FX5UCPU_SD_FX5_4AD_ADP_1	6/4/2021 2:38:23 AM	
M+FX5UCPU_SD_FX5_4AD_ADP_2	6/4/2021 2:39:12 AM	
M+FX5UCPU_SD_FX5_4AD_ADP_3	6/4/2021 2:39:50 AM	
M+FX5UCPU_SD_FX5_4AD_ADP_4	6/4/2021 2:40:37 AM	
M+FX5UCPU_SD_FX5_4AD_PT_ADP_1	7/5/2019 4:12:59 AM	
M+FX5UCPU_SD_FX5_4AD_PT_ADP_2	7/5/2019 4:13:24 AM	
M+FX5UCPU_SD_FX5_4AD_PT_ADP_3	7/5/2019 4:13:48 AM	
M+FX5UCPU_SD_FX5_4AD_PT_ADP_4	7/5/2019 4:14:17 AM	

Workspace Name :
Project Name :
Title :

Data Name	Last Change	Title
M+FX5UCPU_SD_FX5_4AD_TC_ADP_1	7/5/2019 4:12:53 AM	
M+FX5UCPU_SD_FX5_4AD_TC_ADP_2	7/5/2019 4:13:18 AM	
M+FX5UCPU_SD_FX5_4AD_TC_ADP_3	7/5/2019 4:13:42 AM	
M+FX5UCPU_SD_FX5_4AD_TC_ADP_4	7/5/2019 4:14:10 AM	
M+FX5UCPU_SD_FX5_4A_ADP_1_AD	6/4/2021 2:38:46 AM	
M+FX5UCPU_SD_FX5_4A_ADP_1_DA	6/4/2021 2:39:03 AM	
M+FX5UCPU_SD_FX5_4A_ADP_2_AD	6/4/2021 2:39:37 AM	
M+FX5UCPU_SD_FX5_4A_ADP_2_DA	6/4/2021 2:39:44 AM	
M+FX5UCPU_SD_FX5_4A_ADP_3_AD	6/4/2021 2:40:17 AM	
M+FX5UCPU_SD_FX5_4A_ADP_3_DA	6/4/2021 2:40:24 AM	
M+FX5UCPU_SD_FX5_4A_ADP_4_AD	6/4/2021 2:41:04 AM	
M+FX5UCPU_SD_FX5_4A_ADP_4_DA	6/4/2021 2:41:11 AM	
M+FX5UCPU_SD_FX5_4DA_ADP_1	6/4/2021 2:38:36 AM	
M+FX5UCPU_SD_FX5_4DA_ADP_2	6/4/2021 2:39:24 AM	
M+FX5UCPU_SD_FX5_4DA_ADP_3	6/4/2021 2:40:09 AM	
M+FX5UCPU_SD_FX5_4DA_ADP_4	6/4/2021 2:40:52 AM	
M+FX5UCPU_SD_Analog_Adapter	6/4/2021 2:38:15 AM	
M+FX5UCPU_SD_Analog_Input	6/4/2021 2:37:57 AM	
M+FX5UCPU_SD_Analog_Output	6/4/2021 2:38:05 AM	
M+FX5UCPU_SD_CPU_Module_Built_In_Analog	6/4/2021 2:37:47 AM	
M+FX5UCPU_SD_DataLogging_Setting	7/5/2019 4:11:01 AM	
M+FX5UCPU_SD_Data_Logging_Function	2/25/2021 10:16:21 PM	
M+FX5UCPU_SD_FX_Compatible_Area	2/6/2023 6:15:26 PM	
M+FX5UCPU_Module_Information	7/5/2019 4:11:16 AM	
M+FX5UCPU_SD_FX_Dedicated	6/4/2021 12:22:56 AM	
M+FX5UCPU_SD_HighSpeedComparison	7/5/2019 4:11:33 AM	
M+FX5UCPU_SD_HighSpeedCounter	7/5/2019 4:11:28 AM	
M+FX5UCPU_SD_PWM	7/5/2019 4:11:45 AM	
M+FX5UCPU_SD_PulseWidthMeasurement	7/5/2019 4:11:38 AM	
M+FX5UCPU_SD_HighSpeedInOut	6/4/2021 12:22:56 AM	
M+FX5UCPU_SD_Latch_Area	2/25/2021 10:16:25 PM	
M+FX5UCPU_SD_Position	7/5/2019 4:12:03 AM	
M+FX5UCPU_SD_Inverter	2/25/2021 10:08:05 PM	
M+FX5UCPU_SD_MC_Protocol	2/25/2021 10:05:27 PM	
M+FX5UCPU_SD_MODBUS	2/25/2021 10:05:31 PM	
M+FX5UCPU_SD_NN_Network	2/25/2021 10:08:09 PM	
M+FX5UCPU_SD_Parallel_Link	2/25/2021 10:11:13 PM	

Workspace Name :
Project Name :
Title :

Data Name	Last Change	Title
M+FX5UCPU_SD_Predefined_Protocol	2/25/2021 10:13:58 PM	
M+FX5UCPU_SD_SerialComInfo	7/5/2019 4:14:39 AM	
M+FX5UCPU_SD_SerialComMon	7/5/2019 4:14:47 AM	
M+FX5UCPU_SD_SerialComSettingInfo	7/5/2019 4:14:54 AM	
M+FX5UCPU_SD_SerialCom	6/4/2021 12:22:56 AM	
M+FX5UCPU_Time	7/5/2019 4:10:32 AM	
M+FX5UCPU_Scan_Time	6/4/2021 12:22:56 AM	
M+FX5UCPU_SD	2/6/2023 6:15:15 PM	
M+FX5UCPU_SD_Diagnostics1	7/5/2019 4:16:26 AM	
M+FX5UCPU_SD_Diagnostics2	7/5/2019 4:16:32 AM	
M+FX5UCPU_SD_LinkScanTime	7/5/2019 4:16:21 AM	
M+FX5UCPU_SD_Basic	7/26/2022 7:18:33 PM	
M+FX5UCPU_SD_PredefinedProtocol	7/5/2019 4:16:08 AM	
M+FX5UCPU_SD_Time_SynchronizeTimeExecution	2/25/2021 10:21:00 PM	
M+FX5UCPU_SD_EtherPort	6/4/2021 12:22:56 AM	
M+FX5UCPU_SD_SimplePLC	1/21/2022 12:00:42 AM	
M+FX5UCPU_Data_Backup_Restoration_Function	7/5/2019 4:09:16 AM	
M+FX5UCPU_Extended_File_Register_Function	7/5/2019 4:09:27 AM	
M+FX5UCPU_Firmware_Update_Function	7/5/2019 4:05:18 AM	
M+FX5UCPU_Latch_Area	7/5/2019 4:09:21 AM	
M+FX5UCPU_SM_FX5_4AD_ADP_1	6/4/2021 2:31:05 AM	
M+FX5UCPU_SM_FX5_4AD_ADP_2	6/4/2021 2:32:46 AM	
M+FX5UCPU_SM_FX5_4AD_ADP_3	6/4/2021 2:35:39 AM	
M+FX5UCPU_SM_FX5_4AD_ADP_4	6/4/2021 2:36:31 AM	
M+FX5UCPU_SM_FX5_4AD_PT_ADP_1	7/5/2019 4:07:21 AM	
M+FX5UCPU_SM_FX5_4AD_PT_ADP_2	7/5/2019 4:07:46 AM	
M+FX5UCPU_SM_FX5_4AD_PT_ADP_3	7/5/2019 4:08:10 AM	
M+FX5UCPU_SM_FX5_4AD_PT_ADP_4	7/5/2019 4:08:42 AM	
M+FX5UCPU_SM_FX5_4AD_TC_ADP_1	7/5/2019 4:07:15 AM	
M+FX5UCPU_SM_FX5_4AD_TC_ADP_2	7/5/2019 4:07:40 AM	
M+FX5UCPU_SM_FX5_4AD_TC_ADP_3	7/5/2019 4:08:04 AM	
M+FX5UCPU_SM_FX5_4AD_TC_ADP_4	7/5/2019 4:08:36 AM	
M+FX5UCPU_SM_FX5_4A_ADP_1_AD	6/4/2021 2:32:24 AM	
M+FX5UCPU_SM_FX5_4A_ADP_1_DA	2/25/2021 7:10:17 PM	
M+FX5UCPU_SM_FX5_4A_ADP_2_AD	6/4/2021 2:35:22 AM	
M+FX5UCPU_SM_FX5_4A_ADP_2_DA	2/25/2021 7:19:26 PM	
M+FX5UCPU_SM_FX5_4A_ADP_3_AD	6/4/2021 2:36:13 AM	

Workspace Name :
Project Name :
Title :

Data Name	Last Change	Title
M+FX5UCPU_SM_FX5_4A_ADP_3_DA	2/25/2021 7:23:35 PM	
M+FX5UCPU_SM_FX5_4A_ADP_4_AD	6/4/2021 2:36:54 AM	
M+FX5UCPU_SM_FX5_4A_ADP_4_DA	2/25/2021 7:27:46 PM	
M+FX5UCPU_SM_FX5_4DA_ADP_1	7/5/2019 4:07:28 AM	
M+FX5UCPU_SM_FX5_4DA_ADP_2	7/5/2019 4:07:52 AM	
M+FX5UCPU_SM_FX5_4DA_ADP_3	7/5/2019 4:08:16 AM	
M+FX5UCPU_SM_FX5_4DA_ADP_4	6/4/2021 12:22:58 AM	
M+FX5UCPU_SM_Analog_Adapter	6/4/2021 2:30:52 AM	
M+FX5UCPU_SM_Analog_Input	6/4/2021 2:30:33 AM	
M+FX5UCPU_SM_Analog_Output	7/5/2019 4:06:54 AM	
M+FX5UCPU_SM_CPU_Module_Built_In_Analog	6/4/2021 2:30:19 AM	
M+FX5UCPU_SM_DataLogging_Setting	7/5/2019 4:05:30 AM	
M+FX5UCPU_SM_Data_Logging_Function	7/5/2019 4:09:10 AM	
M+FX5UCPU_SM_FX_Compatible_Area	2/6/2023 6:15:21 PM	
M+FX5UCPU_SM_HighSpeedComparison	7/5/2019 4:05:59 AM	
M+FX5UCPU_SM_HighSpeedCounter	8/23/2019 12:41:45 AM	
M+FX5UCPU_SM_PWM	7/5/2019 4:06:22 AM	
M+FX5UCPU_SM_PulseWidthMeasurement	7/5/2019 4:06:08 AM	
M+FX5UCPU_SM_HighSpeedInOut	6/4/2021 12:22:56 AM	
M+FX5UCPU_SM_Position	7/5/2019 4:06:34 AM	
M+FX5UCPU_SM_Inverter	2/25/2021 7:38:08 PM	
M+FX5UCPU_SM_MC_Protocol	2/25/2021 7:38:00 PM	
M+FX5UCPU_SM_MODBUS	2/25/2021 7:38:04 PM	
M+FX5UCPU_SM_NN_Network	2/25/2021 7:38:12 PM	
M+FX5UCPU_SM_Parallel_Link	2/25/2021 7:38:15 PM	
M+FX5UCPU_SM_SerialComError	5/12/2021 1:28:24 AM	
M+FX5UCPU_SM_SerialComFlag	7/5/2019 4:09:04 AM	
M+FX5UCPU_SM_SerialCom	6/4/2021 12:22:56 AM	
M+FX5UCPU_SM	2/6/2023 6:14:58 PM	
M+FX5UCPU	2/6/2023 6:15:03 PM	
EIP_Cognex_Input	3/31/2026 10:17:39 AM	
EIP_Cognex_Output	3/17/2026 6:42:00 PM	
EIP_Motion_Input	11/15/2025 1:54:27 PM	
EIP_Motion_Output	11/13/2025 10:46:44 PM	
VisionFrame	3/31/2026 10:17:56 AM	
DeltaRobot	1/7/2026 12:21:09 AM	

Workspace Name :
Project Name :
Title :

Data Name	Last Change	Title
Position	12/30/2025 9:25:37 PM	
PositionBuffer	1/4/2026 2:55:42 PM	
Device		
Device Comment		
Each Program Device Comment		
Common Device Comment	4/30/2026 12:06:01 PM	
Device Memory	4/30/2026 12:06:40 PM	
MAIN	4/30/2026 12:06:40 PM	
Device Initial Value	4/30/2026 12:05:51 PM	
Parameter		
System Parameter		
FX5UCPU		
CPU Parameter	4/30/2026 12:06:28 PM	
Module Parameter		
Ethernet Port		
485 Serial Port		
High Speed I/O		
Input Response Time		
Analog Input		
Analog Output		
Expansion Board		
Memory Card Parameter		
Module Information		
1[U1]:FX5-ENET/IP	4/30/2026 12:06:33 PM	
Module Parameter		
Remote Password		

[Line Statement List]

Step	Line Statement
(0)	[Title] Manual Signals
	These are used to prevent having to use SET'S and RESET;s in HMI. My preference. get over it
	HMI buttons toggle states. Lamps use direct Input/Output as verifecation. So Secondary command bits are local to save comms size
(300)	[Title] Signal Commands
(371)	[Title] Fault Check
	Main Error check
	If any faults present, SET Vision State to ERROR
(492)	Latch Fault Status to prevent unwanted motion upon startup
(631)	[Title] Unit Conversion, Sort, Frame Setting

[Line Statement List]

Step	Line Statement
(0)	[Title] Manual Signals
	These are used to prevent having to use SET'S and RESET;s in HMI. My preference. get over it
(169)	[Title] Signal Commands
(240)	[Title] Fault Check
	Main Error check
	If any faults present, SET Vision State to ERROR
(361)	Latch Fault Status to prevent unwanted motion upon startup
(496)	[Title] Speed Setting
(703)	[Title] Homing Seq
(811)	[Title] Work Envelope Demo

[Line Statement List]

Step	Line Statement
(0)	[Title] Manual Signals
	These are used to prevent having to use SET'S and RESET;s in HMI. My preference. get over it
(142)	To prevent unwanted motion whenn toggling between auto and manual, unless latch is set
(260)	[Title] Signal Commands
(301)	[Title] Fault Checks
	Main Error check
	If any faults present, SET Vision State to ERROR
(423)	Latch Fault Status to prevent unwanted motion upon startup